

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



DEPARTMENT OF COMPUTER NETWORKS AND TELECOMMUNICATION

REPORT OF PROJECT

Project: Video Streaming System using RTSP/RTP protocols

Lecture:	Lê Hà Minh	
Leader:	Khang Phuc Nguyen	24120068
Members:	Nghia Trong Hoang	24120103
	Trong Minh Pham	24120151

Ho Chi Minh City, 12/2025

Table of Contents

1	Cơ sở lý thuyết	2
1.1	Mô hình kiến trúc và Giao diện Socket	2
1.2	Các giao thức tầng giao vận	2
1.3	Giao thức điều khiển RTSP	3
1.4	Giao thức vận chuyển RTP và Cấu trúc gói tin	4
1.5	Kỹ thuật Video Streaming và Xử lý bộ đệm	4
1.5.1	Định dạng MJPEG và Phân mảnh gói tin (Fragmentation)	4
1.5.2	Cơ chế Bộ đệm (Buffering) và Kiểm soát Jitter	4
1.6	Các mẫu thiết kế phần mềm (Design Patterns)	5
1.6.1	Mẫu thiết kế Builder (Builder Pattern)	5
1.6.2	Mẫu thiết kế Singleton (Singleton Pattern)	5
1.6.3	Mẫu thiết kế Facade (Facade Pattern)	6
1.7	Github Repository	6
2	Tổng quan kiến trúc hoạt động	7
2.1	Sơ đồ triển khai hệ thống (Deployment Diagram)	7
2.2	Định dạng thông điệp RTSP (RTSP Message Format)	7
2.2.1	Định dạng RTSP Request	8
2.2.2	Định dạng RTSP Response	8
3	Implementation	10
3.1	Tổ chức codebase và quy ước build	10
3.2	Common layer: chuẩn hóa giao tiếp và tiện ích dùng chung	11
3.2.1	Socket wrapper và mô hình lỗi	11
3.2.2	RTSPMessage: parse/build theo định dạng dòng	12
3.2.3	RTPPacket và xử lý wrap-around sequence number	12
3.2.4	Builder pattern cho RTPPacket	13
3.3	Server: RTSP state machine và luồng streaming RTP	14
3.3.1	Mô hình đa luồng và quản lý session	14
3.3.2	ServerWorker: RTSP state machine và phân phối handler	14
3.3.3	RTSP State Machine: mô hình chuyển đổi trạng thái nghiêm ngặt	15
3.3.4	RTSP Sequence Diagram: luồng tương tác đầy đủ giữa Client và Server	16



3.3.5	Streaming loop: điều phối FPS, đọc frame, encode và gửi UDP	18
3.3.6	Thuật toán Server Streaming Loop: điều phối chính xác theo FPS	19
3.3.7	Strategy pattern: SD vs HD và phân mảnh payload	21
3.3.8	Giải thích chi tiết về RTP Fragmentation: tại sao cần và cách hoạt động .	22
3.3.9	VideoStream: thuật toán trích xuất frame MJPEG bằng marker JPEG . .	22
3.4	Client: RTSP điều khiển, nhận RTP, ghép frame và buffer/cache	25
3.4.1	RTSPClient: phiên điều khiển và cơ chế “restart”	25
3.4.2	RTPReceiver: nhận UDP và thống kê mất gói	25
3.4.3	Giải thích chi tiết RTP Sequence Number Wrap-around	26
3.4.4	FrameReassembler: thuật toán ghép fragment phức tạp với xử lý out-of-order	27
3.4.5	FrameBuffer: buffer như một “video cache” thread-safe	31
3.4.6	FrameBuffer: vai trò như video cache và cơ chế Producer-Consumer . . .	31
3.4.7	ClientUI (Qt): prebuffer, adaptive playback và cơ chế chống jitter	32
3.5	Ghi nhận triển khai: logging, cấu hình và khả năng mở rộng	35
4	Results	37
4.1	Kết quả kiểm thử đơn vị (Unit Tests)	37
4.2	Chỉ số runtime sẵn có ở Client	38
4.3	Hành vi buffer và prebuffer quan sát từ UI	38
4.4	Hướng dẫn triển khai và tái lập kết quả	39
4.4.1	Thiết lập dự án thông qua <code>setup.py</code>	39
4.4.2	Chạy ứng dụng sau khi build	40
4.4.3	Thu thập dữ liệu và log	41
5	Giao diện của Client	41
6	Video Demo	42
7	Conclusion	43
7.1	Tổng kết đóng góp kỹ thuật theo mã nguồn	43
7.2	Hạn chế hiện tại và đề xuất cải tiến dựa trên bằng chứng mã nguồn	43
7.3	Kết luận	45



Giới thiệu

Với sự phát triển của công nghệ số và nhu cầu giải trí của con người tăng cao, đặc biệt khi cuộc cách mạng 4.0, sự bùng nổ của Internet thì vai trò của Mạng Máy Tính trở nên ngày càng quan trọng và có nhiều ứng dụng thực tiễn chẳng hạn là như là nhắn tin trực tuyến, làm việc trực tuyến và đặc biệt là trao đổi các thông tin, dữ liệu thông quan mạng. Đặc biệt trong số đó thì việc truyền các Video hoặc Ảnh thông qua được sử dụng ngày phổ biến trong các ứng dụng mạng xã hội như Facebook, TikTok và Youtube.

Trong dự án này, nhóm đã tìm hiểu và thực hiện hóa một hệ thống đơn giản cho phép hai ứng dụng mạng có thể giao tiếp với nhau thông qua giao tiếp RTSP/RTP để có thể phát một video trực tuyến. Dự án này được làm với mục đích học tập và hiểu rõ hơn về các giao thức cũng như là cách hoạt động của các ứng dụng trong thực tế, mặt khác, dự án này cũng chỉ là mô phỏng lại cách phát trực tiếp một video giữa các ứng dụng nên được xây dựng không quá phức tạp và tối ưu.

Toàn bộ mã nguồn của dự án được xây dựng bằng ngôn ngữ `C++ 17`, sử dụng `CMake` để có thể build trên cả Window và Linux, cũng như là sử dụng các thư viện `winsock2` và `POSIX sockets` để tạo các socket cho việc giao tiếp giữa các ứng dụng. Bên cạnh đó, nhóm cũng đã áp dụng các kiến thức về `Design Pattern` cho dự án với mục đích là mã nguồn của dự án có thể tái sử dụng, sạch sẽ và dễ đọc, dễ sửa lỗi cũng như là tối ưu trong quá trình hoạt động.

Để có thể có một giao diện cho ứng dụng của bên phía `client` thì nhóm đã sử dụng `Qt6` để tạo ra một giao diện đơn giản và dễ sử dụng cho người dùng. Toàn bộ quá trình, giao thức, phản hồi và yêu cầu của `server` và `client` được dựa trên toàn bộ yêu cầu được đính kèm trong file `Socket_2526` của thầy Lê Hà Minh [1]. Nhóm xin chân thành cảm ơn thầy đã hỗ trợ và cung cấp tài liệu để nhóm có thể hoàn thành đồ án này một cách tốt đẹp nhất.

Nhóm Trưởng

(Chữ ký và họ tên)

Khang

Nguyễn Phúc Khang

Ho Chi Minh City, 12/2025

1 Cơ sở lý thuyết

Hệ thống truyền phát video trong đề án được xây dựng dựa trên sự kết hợp chặt chẽ giữa kiến trúc mạng máy tính kinh điển và các kỹ thuật xử lý dữ liệu đa phương tiện thời gian thực, tuân thủ theo các yêu cầu của môn học [1]. Phần này sẽ trình bày chi tiết về các giao thức, kỹ thuật socket nâng cao và các mẫu thiết kế phần mềm được áp dụng.

1.1 Mô hình kiến trúc và Giao diện Socket

Hệ thống vận hành dựa trên mô hình **Client-Server**, trong đó trách nhiệm được phân chia rõ ràng giữa hai thực thể. **Server** đóng vai trò là bên cung cấp dịch vụ thụ động (*Passive Open*), chịu trách nhiệm lưu trữ tài nguyên video, quản lý các phiên làm việc và điều phối luồng dữ liệu. Ngược lại, **Client** đóng vai trò chủ động (*Active Open*), khởi tạo các yêu cầu kết nối và thực hiện tái tạo hình ảnh từ dữ liệu nhận được.

Để hiện thực hóa cơ chế giao tiếp này, dự án sử dụng **Socket API** như một giao diện lập trình cốt lõi [2]. Socket đóng vai trò là điểm cuối trong liên kết hai chiều giữa hai chương trình chạy trên mạng. Tùy thuộc vào yêu cầu về độ tin cậy hay tốc độ của từng loại dữ liệu, hệ thống sẽ khởi tạo loại socket phù hợp: **Stream Socket** (sử dụng TCP) cho kênh điều khiển nhằm đảm bảo tính chính xác tuyệt đối của lệnh, hoặc **Datagram Socket** (sử dụng UDP) cho kênh dữ liệu để tối ưu hóa độ trễ thời gian thực. Bên cạnh đó, các tùy chọn nâng cao như `SO_REUSEADDR` cũng được áp dụng để quản lý trạng thái cổng kết nối hiệu quả, tránh các lỗi khóa tài nguyên thường gặp trong lập trình mạng UNIX [2].

1.2 Các giao thức tầng giao vận

Việc lựa chọn giao thức tầng giao vận là quyết định kỹ thuật quan trọng nhất ảnh hưởng đến hiệu năng của ứng dụng streaming. Thay vì chỉ sử dụng một giao thức đơn lẻ, hệ thống áp dụng cơ chế lai (Hybrid) để tận dụng ưu điểm của cả TCP và UDP theo kiến trúc nền tảng về mạng máy tính [3]. Sự khác biệt và phạm vi áp dụng của hai giao thức này trong dự án được tóm tắt tại Bảng 1.

Table 1: So sánh và phạm vi áp dụng của TCP và UDP trong hệ thống

Giao thức	Đặc điểm kỹ thuật	Áp dụng trong dự án
TCP	Hướng kết nối, đảm bảo tin cậy, có kiểm soát luồng và tắc nghẽn, độ trễ cao hơn do cơ chế truyền lại (retransmission).	Kênh RTSP: Truyền tải các lệnh điều khiển quan trọng như SETUP, PLAY.
UDP	Phi kết nối, không đảm bảo tin cậy, header nhỏ (8 byte), độ trễ thấp, hỗ trợ truyền thời gian thực.	Kênh RTP: Vận chuyển dữ liệu video (JPEG). Chấp nhận mất gói để giữ độ trễ thấp.

1.3 Giao thức điều khiển RTSP

Real-Time Streaming Protocol (RTSP), được định nghĩa trong RFC 2326 [4], hoạt động như một "bộ điều khiển từ xa" qua mạng cho các máy chủ đa phương tiện. Điểm khác biệt cốt lõi của RTSP so với HTTP là tính chất *lưu trạng thái* (stateful). Server duy trì một phiên làm việc thông qua mã định danh **Session ID**, cho phép Server theo dõi trạng thái hiện tại của Client để phản hồi hợp lý.

Quá trình chuyển đổi trạng thái của hệ thống tuân theo quy trình nghiêm ngặt: từ trạng thái **INIT** (khởi tạo), chuyển sang **READY** (sẵn sàng) sau khi thiết lập tài nguyên thành công, và cuối cùng là **PLAYING** (đang phát) khi luồng dữ liệu bắt đầu được truyền tải. Các phương thức điều khiển chính được mô tả trong Bảng 2.

Table 2: Các phương thức RTSP và chức năng tương ứng

Phương thức	Mô tả chức năng
SETUP	Khởi tạo phiên làm việc. Client gửi cổng nhận RTP (<i>client_port</i>), Server phản hồi với <i>Session ID</i> và cổng truyền của Server.
PLAY	Yêu cầu Server bắt đầu truyền dữ liệu RTP. Server chuyển trạng thái sang PLAYING.
PAUSE	Tạm dừng luồng dữ liệu. Server ngừng gửi gói tin nhưng vẫn giữ nguyên phiên làm việc và trạng thái kết nối.
TEARDOWN	Kết thúc phiên làm việc. Server đóng kết nối, giải phóng bộ đệm và thu hồi Session ID.

1.4 Giao thức vận chuyển RTP và Cấu trúc gói tin

Trong khi RTSP chịu trách nhiệm điều khiển, **Real-time Transport Protocol (RTP)** theo tiêu chuẩn RFC 3550 [5] đảm nhiệm vai trò vận chuyển dữ liệu video thực tế. Do đặc thù của giao thức UDP là không đảm bảo thứ tự, RTP bổ sung một phần tiêu đề (Header) dài 12 byte vào trước mỗi gói dữ liệu để cung cấp thông tin tái đồng bộ.

Ba trường thông tin quan trọng nhất trong RTP Header bao gồm: **Sequence Number** (16 bit) giúp bên nhận phát hiện mất gói và sắp xếp lại các gói tin bị đảo lộn; **Timestamp** (32 bit) đóng vai trò nhận thời gian để đồng bộ hóa tốc độ hiển thị khung hình, đảm bảo video không bị tua nhanh hoặc chậm bất thường; và **Marker Bit** (1 bit) dùng để đánh dấu gói tin cuối cùng của một khung hình, hỗ trợ quá trình tái lắp ráp dữ liệu phân mảnh.

1.5 Kỹ thuật Video Streaming và Xử lý bộ đệm

Để đảm bảo khả năng truyền tải video mượt mà trên môi trường mạng biến động, hệ thống áp dụng các kỹ thuật xử lý dữ liệu nâng cao liên quan đến định dạng nén, phân mảnh và cơ chế đệm.

1.5.1 Định dạng MJPEG và Phân mảnh gói tin (Fragmentation)

Dự án sử dụng chuẩn nén **MJPEG (Motion JPEG)**, nơi mỗi khung hình video là một bức ảnh JPEG độc lập sử dụng kỹ thuật nén nội khung (intra-frame compression) [6]. Phương pháp này tuân thủ RFC 2435 [7] và mang lại khả năng chịu lỗi cao, vì việc mất một gói tin chỉ ảnh hưởng cục bộ đến khung hình hiện tại mà không gây lỗi dây chuyền cho các khung hình kế tiếp.

Tuy nhiên, kích thước của một khung hình video (đặc biệt ở độ phân giải HD) thường vượt quá giới hạn **MTU (Maximum Transmission Unit)** của mạng Ethernet (thông thường là 1500 byte). Để giải quyết vấn đề này, dự án áp dụng kỹ thuật **Phân mảnh cấp ứng dụng** (Application-Level Fragmentation). Một khung hình lớn sẽ được chia nhỏ thành nhiều gói RTP có kích thước tối đa 1400 byte. Hệ thống sử dụng mẫu thiết kế **Strategy Pattern** [8] để linh hoạt chuyển đổi thuật toán xử lý giữa video SD (có thể không cần phân mảnh) và video HD (bắt buộc phân mảnh), đảm bảo tính tương thích và hiệu năng cao nhất.

1.5.2 Cơ chế Bộ đệm (Buffering) và Kiểm soát Jitter

Hiện tượng **Jitter** (sự biến thiên độ trễ gói tin) là nguyên nhân chính gây ra tình trạng giật hình trong streaming [3]. Để khắc phục, Client được thiết kế hoạt động theo mô hình **Producer-**

Consumer với một **Frame Buffer** trung gian. Luồng nhận mạng đóng vai trò Producer, liên tục tái lắp ráp các gói tin RTP và đẩy khung hình hoàn chỉnh vào hàng đợi. Luồng hiển thị giao diện đóng vai trò Consumer, lấy khung hình ra để hiển thị theo định kỳ.

Đặc biệt, kỹ thuật **Pre-buffering** (Tiền tải) được áp dụng nhằm yêu cầu Client phải tích lũy đủ một lượng khung hình nhất định (ngưỡng N frames) trước khi bắt đầu phát. Cơ chế này giúp tạo ra một vùng đệm an toàn, triệt tiêu các dao động tức thời của mạng và đảm bảo trải nghiệm xem video liên tục.

1.6 Các mẫu thiết kế phần mềm (Design Patterns)

Để đảm bảo mã nguồn có khả năng mở rộng, dễ bảo trì và tuân thủ các nguyên lý lập trình hướng đối tượng, dự án đã áp dụng một số mẫu thiết kế phần mềm kinh điển được đề xuất bởi Gamma và cộng sự [8]. Việc áp dụng các mẫu này giúp giải quyết các vấn đề phổ biến trong việc khởi tạo đối tượng, quản lý trạng thái toàn cục và đơn giản hóa các giao diện phức tạp.

1.6.1 Mẫu thiết kế Builder (Builder Pattern)

Trong giao thức RTP, việc khởi tạo một gói tin (**RTPPacket**) đòi hỏi thiết lập rất nhiều tham số chi tiết cho phần tiêu đề (Header) như: phiên bản (Version), chỉ số đếm (Sequence Number), nhãn thời gian (Timestamp), bit đánh dấu (Marker), và loại tải (Payload Type). Việc sử dụng một hàm khởi tạo (Constructor) với quá nhiều tham số sẽ khiến mã nguồn khó đọc và dễ gây lỗi.

Để giải quyết vấn đề này, dự án áp dụng mẫu thiết kế **Builder Pattern**. Cụ thể, lớp **RTPPacketBuilder** (nằm trong thư mục `common/src/rtp`) được xây dựng để tách biệt quá trình khởi tạo phức tạp khỏi biểu diễn của đối tượng. Mẫu thiết kế này cho phép thiết lập từng thuộc tính của gói tin một cách tuần tự và rõ ràng thông qua chuỗi các phương thức (method chaining) trước khi gọi phương thức `build()` để tạo ra đối tượng gói tin hoàn chỉnh.

1.6.2 Mẫu thiết kế Singleton (Singleton Pattern)

Trong một hệ thống phân tán, có những thành phần cần đảm bảo tính duy nhất và có thể truy cập toàn cục từ bất kỳ đâu trong chương trình, điển hình là bộ ghi nhật ký (Logger) và bộ quản lý cấu hình (Configuration Manager).

Dự án áp dụng mẫu thiết kế **Singleton Pattern** cho các lớp **Logger** và **Config** (trong thư mục `common/src/utils`). Mẫu thiết kế này đảm bảo rằng chỉ có duy nhất một thể hiện (instance)

của lớp **Logger** được tạo ra trong suốt vòng đời ứng dụng, giúp đồng bộ hóa việc ghi log từ nhiều luồng (thread) khác nhau vào một tệp tin hoặc màn hình console duy nhất, tránh tình trạng xung đột tài nguyên. Tương tự, **Config** đóng vai trò là kho chứa thông tin cấu hình trung tâm (như địa chỉ IP, cổng, kích thước bộ đệm) mà mọi thành phần khác đều có thể truy xuất.

1.6.3 Mẫu thiết kế Facade (Facade Pattern)

Lập trình mạng trên hệ điều hành thường yêu cầu tương tác với các API cấp thấp phức tạp của thư viện Berkeley Sockets (như `sockaddr_in`, `bind`, `listen`, `setsockopt`). Việc gọi trực tiếp các hàm C này rải rác trong mã nguồn sẽ làm giảm tính trừu tượng và khó chuyển đổi giữa các nền tảng (như Windows và Linux).

Để khắc phục, lớp **Socket** (trong thư mục `common/src/network`) được thiết kế như một **Facade** (mặt tiền). Lớp này cung cấp một giao diện lập trình hướng đối tượng đơn giản và thống nhất (với các phương thức như `send()`, `recv()`, `bind()`) để che giấu sự phức tạp của các lời gọi hệ thống bên dưới. Nhờ đó, các tầng cao hơn của ứng dụng (như RTSP Server hay RTP Sender) chỉ cần làm việc với đối tượng **Socket** mà không cần quan tâm đến chi tiết cài đặt của hệ điều hành.

Table 3: Tổng hợp các mẫu thiết kế được áp dụng trong cấu trúc mã nguồn

Mẫu thiết kế	Loại (Category)	Áp dụng thực tế trong dự án
Builder	Creational (Khởi tạo)	RTPPacketBuilder : Xây dựng gói tin RTP phức tạp từng bước một.
Singleton	Creational (Khởi tạo)	Logger , Config : Quản lý tài nguyên duy nhất (log file, cấu hình) toàn cục.
Facade	Structural (Cấu trúc)	Socket : Đóng gói và đơn giản hóa các thao tác với thư viện socket cấp thấp của hệ điều hành.
Strategy	Behavioral (Hành vi)	EncodingStrategy : Linh hoạt thay đổi thuật toán phân mảnh cho video SD và HD (đã đề cập ở phần trước).

1.7 Github Repository

Dự án được lưu trữ trên Github tại địa chỉ <https://github.com/khang1108/rocketProgramming>. Nhóm đã có đẩy code lên và cũng như là tạo các bản **release** cho các nền tảng Windows, macOS và Linux.

2 Tổng quan kiến trúc hoạt động

2.1 Sơ đồ triển khai hệ thống (Deployment Diagram)

Hình 1 minh họa kiến trúc triển khai của hệ thống trên hai máy vật lý riêng biệt. Server chạy tiến trình `./server 8554`, lắng nghe các yêu cầu RTSP trên cổng TCP 8554. Client chạy tiến trình `./client`, kết nối đến Server để điều khiển phát video và nhận luồng dữ liệu RTP qua UDP. Mỗi thành phần trong sơ đồ đại diện cho một module chức năng cụ thể trong mã nguồn, từ việc xử lý giao thức RTSP, đọc và mã hóa video, đóng gói RTP, đến tái lắp ráp và hiển thị khung hình.

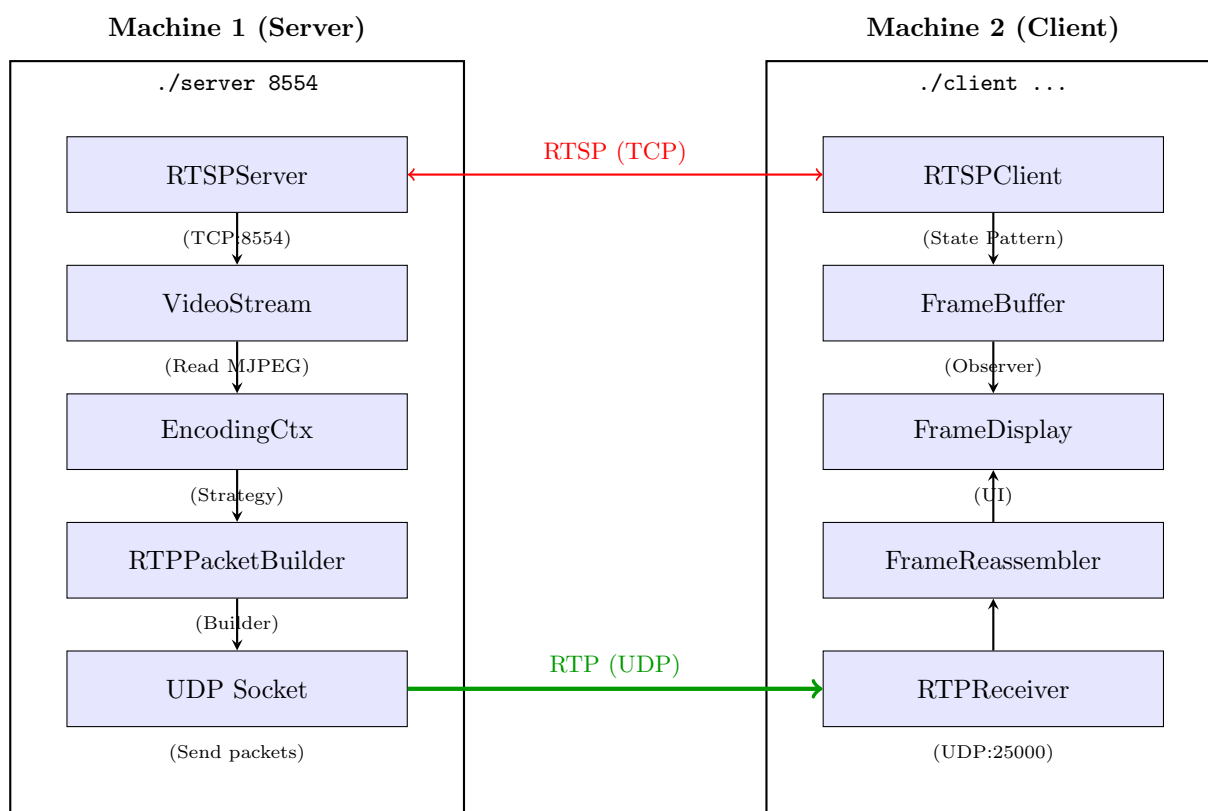


Figure 1: Sơ đồ triển khai hệ thống video streaming phân tán

2.2 Định dạng thông điệp RTSP (RTSP Message Format)

Giao thức RTSP sử dụng cú pháp văn bản dạng ASCII, tương tự HTTP, để dễ dàng đọc và gỡ lỗi. Mỗi thông điệp RTSP bao gồm một dòng bắt đầu (Request Line hoặc Status Line), các

dòng tiêu đề (Headers), và một dòng trống (`\r\n`) để đánh dấu kết thúc thông điệp.

2.2.1 Định dạng RTSP Request

Một yêu cầu RTSP từ Client gửi đến Server có cấu trúc như sau (xem Hình 2):

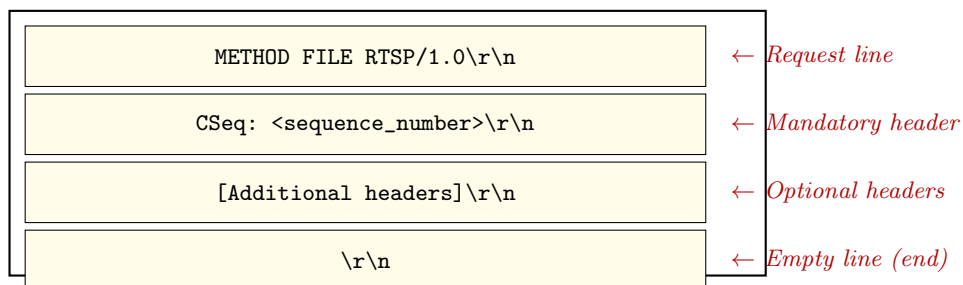


Figure 2: Cấu trúc thông điệp RTSP Request

Trong đó:

- **METHOD:** Phương thức điều khiển (ví dụ: SETUP, PLAY, PAUSE, TEARDOWN)
- **FILE:** Đường dẫn đến tài nguyên video trên Server (ví dụ: movie.mjpeg)
- **RTSP/1.0:** Phiên bản giao thức
- **CSeq:** Số thứ tự yêu cầu (bắt buộc), dùng để đối chiếu với phản hồi
- **Additional headers:** Các tiêu đề tùy chọn khác như Session, Transport

2.2.2 Định dạng RTSP Response

Server phản hồi lại Client với thông điệp có cấu trúc như Hình 3:

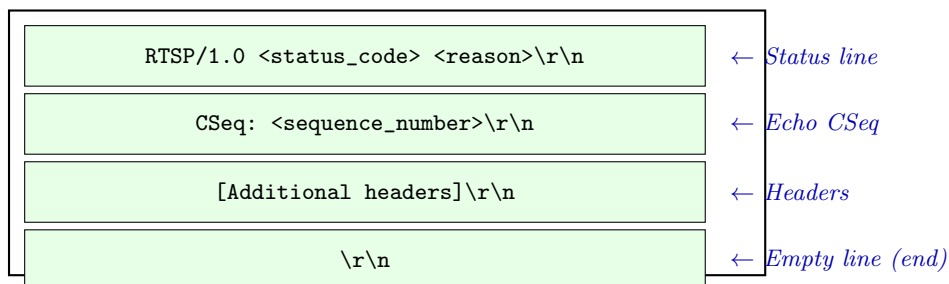


Figure 3: Cấu trúc thông điệp RTSP Response

Trong đó:



- **status_code**: Mã trạng thái HTTP-like (ví dụ: 200 OK, 404 Not Found)
- **reason**: Chuỗi mô tả trạng thái (ví dụ: OK, Not Found)
- **CSeq**: Phản chiếu lại số thứ tự từ yêu cầu để Client đối chiếu
- **Headers**: Các thông tin bổ sung như Session, Transport, Content-Length

Việc sử dụng **CSeq** (Command Sequence) trong cả Request và Response đảm bảo tính toàn vẹn của giao tiếp, đặc biệt trong môi trường mạng bất đồng bộ, giúp Client xác định chính xác phản hồi nào tương ứng với yêu cầu nào khi có nhiều yêu cầu được gửi đồng thời.

3 Implementation

Mục tiêu của phần này là mô tả đầy đủ cách hệ thống được hiện thực ở mức mã nguồn: cách tổ chức module, cấu trúc lớp, luồng điều khiển của RTSP/RTP, cơ chế đa luồng, thuật toán đóng gói–giải gói RTP (bao gồm phân mảnh cho frame lớn), thuật toán trích xuất frame MJPEG, và cơ chế “buffer như một lớp cache” ở phía Client để ổn định trải nghiệm xem.

3.1 Tổ chức codebase và quy ước build

Dự án được tách thành ba khối chính: **Server**, **Client** và **Common**. Common đóng vai trò thư viện dùng chung cho cả hai phía (network wrapper, RTSP message parsing/building, RTP packet, tiện ích). Việc chia tách này giúp cô lập phụ thuộc (đặc biệt là Qt chỉ xuất hiện ở Client), giảm vòng lặp biên dịch, và làm rõ ranh giới trách nhiệm.

Table 4: Phân rã module và phạm vi trách nhiệm

Module	Trách nhiệm chính	Ví dụ file trọng điểm
<code>common/</code>	Thư viện dùng chung (Socket, RTSP, RTP, Logger/Timer/Config)	<code>common/src/network/Socket.cpp</code> , <code>common/src/rtp/RTPPacket.cpp</code>
<code>server/</code>	RTSP server + luồng streaming RTP + đọc MJPEG	<code>server/src/network/RTSPServer.cpp</code> , <code>server/src/network/ServerWorker.cpp</code> , <code>server/src/video/VideoStream.cpp</code>
<code>client/</code>	RTSP client + nhận RTP + ghép frame + buffer/cache + Qt UI	<code>client/src/network/RTSPClient.cpp</code> , <code>client/src/rtp/RTPReceiver.cpp</code> , <code>client/src/buffer/FrameBuffer.cpp</code> , <code>client/src/ui/ClientUI.cpp</code>

Build system sử dụng CMake và tách các target theo thư mục con; Qt6 chỉ được yêu cầu ở cấp root để build Client. Các include directory được cấu hình để cả Server/Client truy cập được Common.

```
1 find_package(Qt6 REQUIRED COMPONENTS Core Widgets Network Multimedia MultimediaWidgets)
2
3 include_directories(
4     ${CMAKE_SOURCE_DIR}/common/include
5     ${CMAKE_SOURCE_DIR}/client/include
6     ${CMAKE_SOURCE_DIR}/server/include
7 )
```

```
8  
9 add_subdirectory(common)  
10 add_subdirectory(client)  
11 add_subdirectory(server)
```

Listing 1: CMake cấu hình Qt6 và chia subdirectory

3.2 Common layer: chuẩn hóa giao tiếp và tiện ích dùng chung

3.2.1 Socket wrapper và mô hình lỗi

Thay vì gọi trực tiếp POSIX/WinSock API, hệ thống xây dựng lớp `Socket` như một “adapter” thống nhất cho TCP/UDP. Lớp này đóng gói các thao tác then chốt: `bind()/listen()/accept()` cho TCP server, `connect()` cho TCP client, `send()/recv()` cho RTSP qua TCP, và `sendTo()/recvFrom()` cho RTP qua UDP. Điểm quan trọng là lớp `Socket` chủ động chuyển các lỗi hệ thống về exception ở mức C++ (`SocketException`, `SocketTimeout`), giúp code xử lý luồng điều khiển rõ ràng hơn (đặc biệt trong vòng lặp nhận RTP).

```
1 int Socket::send(const uint8_t* data, size_t length) {  
2     size_t totalSent = 0;  
3     while (totalSent < length) {  
4         int bytes = ::send(sockfd_, (const char*)(data + totalSent),  
5                             length - totalSent, 0);  
6         if (bytes == SOCKET_ERROR) {  
7             // map error -> SocketTimeout / SocketException  
8             ...  
9         }  
10        totalSent += bytes;  
11    }  
12    return (int)totalSent;  
13 }
```

Listing 2: Vòng lặp `send()` đảm bảo gửi đủ dữ liệu RTSP qua TCP

Với RTP (UDP), socket được đặt timeout ngắn để thread nhận không bị block vĩnh viễn và có thể thoát khi `running_` chuyển trạng thái.

```
1 socket_->setTimeout(5); // milliseconds (UDP)
```

```
2...
3try {
4    int bytes = socket_>recvFrom(buffer, sizeof(buffer), senderAddr);
5    ...
6} catch (const SocketTimeout&) {
7    continue; // loop again to check running_
8}
```

Listing 3: Nhận RTP với timeout để kiểm tra cờ dừng định kỳ

3.2.2 RTSPMessage: parse/build theo định dạng dòng

Thay vì parse “thủ công” rả rác, hệ thống gom chuẩn hóa RTSP ở lớp `RTSPMessage`. Lớp này cung cấp: (1) `buildRequest/buildResponse` để sinh chuỗi RTSP chuẩn CRLF, (2) `parseRequest/parseResponse` để tách request line/status line và headers vào `std::map`, (3) quy ước đọc `CSeq` thành số nguyên để dùng trực tiếp.

```
1std::string RTSPMessage::buildRequest(const std::string& method,
2                                     const std::string url,
3                                     int cseq,
4                                     const std::map<std::string, std::string>& headers)
5{
6    std::stringstream ss;
7    ss << method << " " << url << " RTSP/1.0\r\n";
8    ss << "CSeq: " << cseq << "\r\n";
9    for (const auto& pair : headers) ss << pair.first << ": " << pair.second << "\r\n";
10    ss << "\r\n";
11    return ss.str();
12}
```

Listing 4: Sinh request RTSP chuẩn CRLF và header CSeq

3.2.3 RTPPacket và xử lý wrap-around sequence number

`RTPPacket` hiện thực header RTP (version, marker, payload type, sequence number, timestamp, SSRC) và payload. Một điểm dễ sai trong RTP là **sequence number 16-bit** sẽ wrap-around;

vì vậy project bổ sung hàm `sequenceDifference()` để đo khoảng cách sequence theo modulo 2^{16} , phục vụ thống kê mất gói ở Client và kiểm soát thứ tự ở Reassembler.

```
1 int32_t RTPPacket::sequenceDifference(uint16_t seq1, uint16_t seq2) {
2     int32_t diff = (int32_t)seq1 - (int32_t)seq2;
3     if (diff > 32768) diff -= 65536;
4     if (diff < -32768) diff += 65536;
5     return diff;
6 }
```

Listing 5: Tính chênh lệch sequence theo modulo để xử lý wrap-around

3.2.4 Builder pattern cho RTPPacket

Để giảm lỗi khi dựng packet và tăng tính “self-documenting”, hệ thống sử dụng Builder pattern: `RTPPacketBuilder` yêu cầu set các trường bắt buộc (`sequenceNumber/timestamp/ssrc/payload`) trước khi `build()`. Nếu thiếu trường, builder ném `std::runtime_error` để fail-fast ngay tại nơi tạo packet.

```
1 RTPPacket RTPPacketBuilder::build() {
2     if (!seqSet_) throw std::runtime_error("Sequence number not set");
3     if (!timestampSet_) throw std::runtime_error("Timestamp not set");
4     if (!ssrcSet_) throw std::runtime_error("SSRC not set");
5     if (!payloadSet_) throw std::runtime_error("Payload not set");
6     RTPPacket packet;
7     packet.setSequenceNumber(sequenceNumber_);
8     packet.setTimestamp(timestamp_);
9     packet.setSSRC(ssrc_);
10    packet.setPayload(payload_);
11    packet.encode(); // serialize header -> raw bytes
12    return packet;
13 }
```

Listing 6: Builder kiểm tra các trường bắt buộc trước khi build RTP packet

3.3 Server: RTSP state machine và luồng streaming RTP

3.3.1 Mô hình đa luồng và quản lý session

`RTSPServer` chạy vòng lặp `accept()` trên TCP listen socket. Mỗi kết nối mới được gán `clientId`, tạo một `ServerWorker` và chạy nó trên một `std::thread` riêng. Danh sách session được bảo vệ bởi `std::mutex` để tránh race khi stop server.

```
1 while (running) {
2     std::unique_ptr<Socket> clientSocket = listenSocket->accept();
3     ClientSession session;
4     session.clientId = nextClientId++;
5     session.worker = std::make_unique<ServerWorker>(session.clientId,
6         std::move(clientSocket));
7     session.workerThread = std::thread(&ServerWorker::run, session.worker.get());
8     activeSession.push_back(std::move(session));
9 }
```

Listing 7: Tạo thread cho từng client trong `RTSPServer::run()`

3.3.2 ServerWorker: RTSP state machine và phân phối handler

Mỗi `ServerWorker` duy trì state nội bộ `INIT/READY/PLAYING/PAUSED`. Vòng lặp `handleRtspRequests()` nhận request RTSP, parse, sau đó dispatch sang `handleSetup/handlePlay/handlePause/handleTeardown`. Việc tách handler giúp mỗi lệnh RTSP có phạm vi trách nhiệm rõ ràng (validate session, đổi state, start/stop thread streaming).

```
1 auto request = RTSPMessage::parseRequest(msg);
2 if (request.method == "SETUP") response = handleSetup(request);
3 else if (request.method == "PLAY") response = handlePlay(request);
4 else if (request.method == "PAUSE") response = handlePause(request);
5 else if (request.method == "TEARDOWN") response = handleTeardown(request);
6 else response = RTSPMessage::buildResponse(400, "Bad Request", request.cseq);
```

Listing 8: Dispatch lệnh RTSP theo method ở `ServerWorker`

3.3.3 RTSP State Machine: mô hình chuyển đổi trạng thái nghiêm ngặt

Hệ thống RTSP hoạt động dựa trên một máy trạng thái hữu hạn (Finite State Machine - FSM) với các quy tắc chuyển đổi nghiêm ngặt. Điều này đảm bảo rằng Client và Server luôn đồng bộ về trạng thái phiên làm việc, tránh các lỗi logic như gửi PAUSE khi chưa PLAY hoặc PLAY khi chưa SETUP.

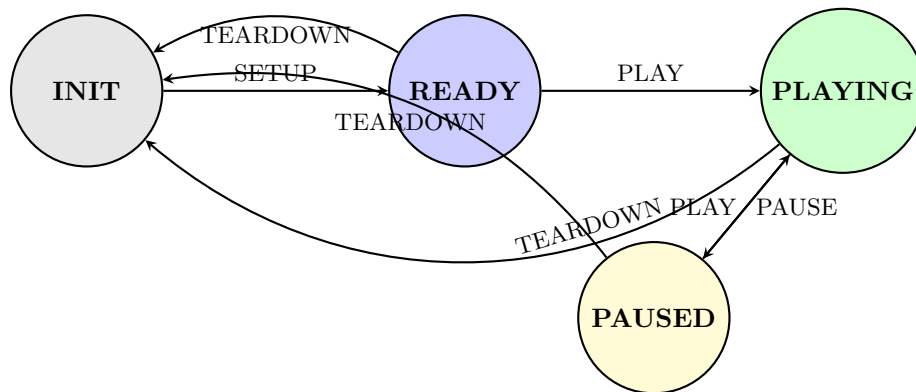


Figure 4: Máy trạng thái RTSP với các chuyển đổi hợp lệ

Ý nghĩa và điều kiện của từng trạng thái được mô tả trong Bảng 5:

Table 5: Các trạng thái RTSP và điều kiện chuyển đổi

Trạng thái	Ý nghĩa	Chuyển đổi hợp lệ
INIT	Trạng thái khởi tạo, chưa có session nào được thiết lập.	Chỉ chấp nhận SETUP.
READY	Session đã được tạo, tài nguyên đã cấp phát, sẵn sàng streaming.	Chấp nhận PLAY (chuyển sang PLAYING) hoặc TEARDOWN (về INIT).
PLAYING	Đang truyền dữ liệu RTP qua UDP.	Chấp nhận PAUSE (chuyển sang PAUSED) hoặc TEARDOWN (về INIT).
PAUSED	Tạm dừng streaming nhưng vẫn giữ session.	Chấp nhận PLAY (tiếp tục streaming) hoặc TEARDOWN (về INIT).

Lý do thiết kế State Machine: Máy trạng thái giúp hệ thống có thể kiểm tra tính hợp lệ của mỗi request dựa trên trạng thái hiện tại. Ví dụ, nếu Client gửi PLAY khi đang ở trạng thái INIT (chưa SETUP), Server sẽ trả về lỗi 455 Method Not Valid in This State. Điều này đảm bảo tính nhất quán của giao thức và tránh các hành vi không xác định.

```
1 std::string ServerWorker::handlePlay(const RTSPRequest& request) {
```

```
2  if (state_ != State::READY && state_ != State::PAUSED) {  
3      return RTSPMessage::buildResponse(455, "Method Not Valid in This State",  
4                                          request.cseq);  
5  }  
6  // ... proceed with PLAY logic  
7  state_ = State::PLAYING;  
8  return RTSPMessage::buildResponse(200, "OK", request.cseq, headers);  
9 }
```

Listing 9: Validation state trước khi xử lý PLAY request

3.3.4 RTSP Sequence Diagram: luồng tương tác đầy đủ giữa Client và Server

Để hiểu rõ hơn về cách RTSP và RTP phối hợp, Hình 5 minh họa một phiên làm việc hoàn chỉnh từ lúc Client kết nối đến khi kết thúc session.

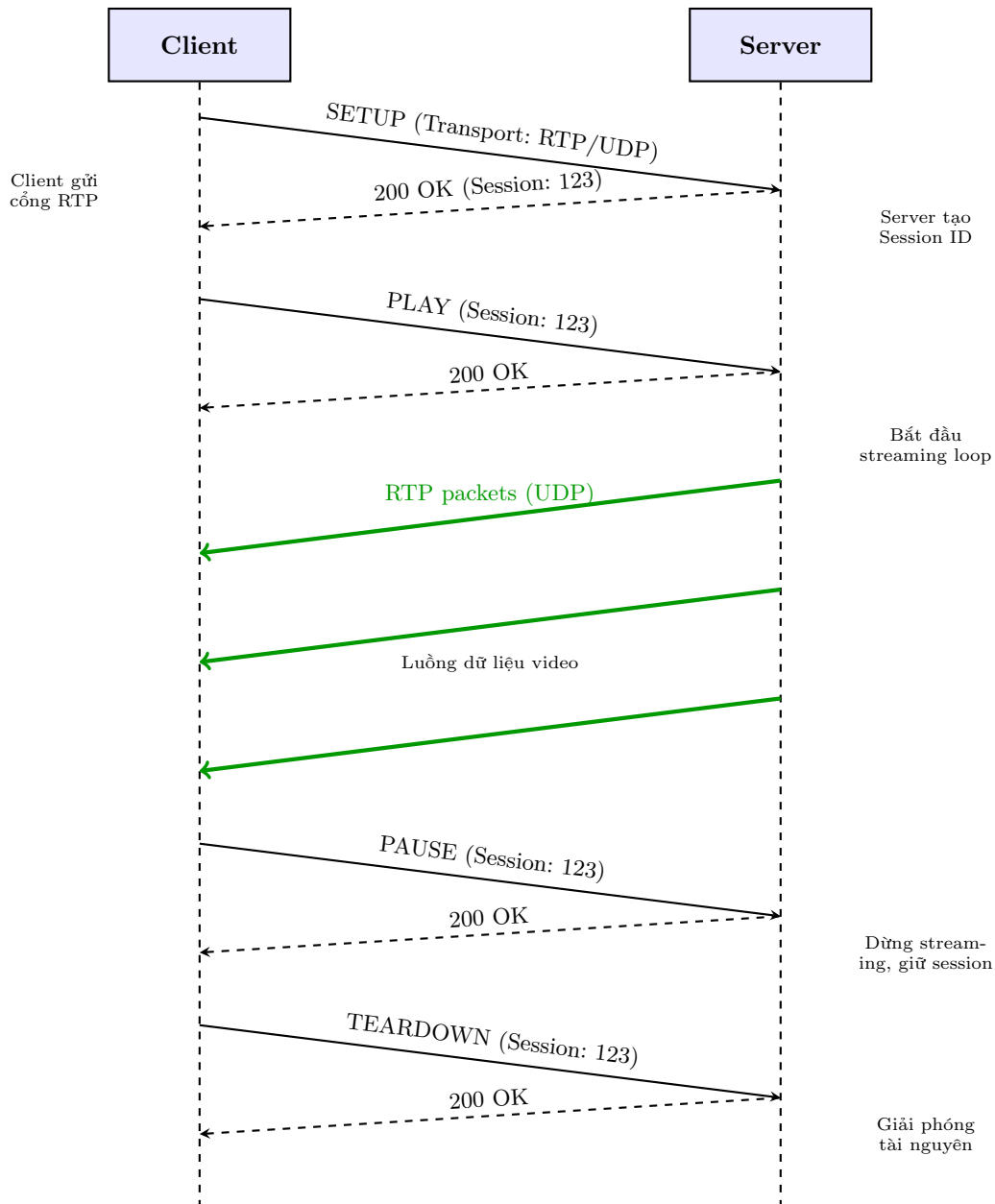


Figure 5: Sequence diagram của một phiên RTSP/RTP hoàn chỉnh

Giải thích chi tiết các bước:

1. **SETUP phase:** Client gửi cổng UDP mà nó sẽ lắng nghe RTP (ví dụ: 25000) trong header Transport. Server phản hồi với **Session ID** (dùng để nhận diện phiên này trong các request sau) và cổng UDP của Server. Đây là bước chuẩn bị tài nguyên.

2. **PLAY phase:** Client gửi PLAY kèm Session ID. Server khởi động background thread để đọc video file, encode thành RTP packets và gửi qua UDP. Lưu ý rằng RTSP response trả về ngay lập tức, không đợi streaming loop chạy.
3. **RTP streaming:** Luồng dữ liệu RTP chảy độc lập qua UDP, không có acknowledgment. Đây là điểm khác biệt quan trọng: control plane (RTSP/TCP) và data plane (RTP/UDP) tách biệt hoàn toàn.
4. **PAUSE:** Dừng streaming loop nhưng không giải phóng session. Client có thể gửi PLAY lại để tiếp tục từ frame hiện tại.
5. **TEARDOWN:** Dọn dẹp hoàn toàn, đóng socket UDP, giải phóng Session ID. Client cần SETUP lại nếu muốn xem tiếp.

3.3.5 Streaming loop: điều phối FPS, đọc frame, encode và gửi UDP

Khi nhận PLAY hợp lệ, Worker khởi động một background thread chạy `streamingLoop()`. Vòng lặp này có ba bước trọng tâm: (1) đọc frame từ `VideoStream`, (2) encode frame thành một hoặc nhiều RTP packet (Strategy), (3) gửi packet qua UDP tới `clientIP_ : clientRTPPort_`. `Timer` được dùng để kiểm soát nhịp gửi theo interval.

```
1 while (streaming_) {  
2     frameTimer_>start();  
3  
4     std::vector<uint8_t> frame = videoStream_>nextFrame();  
5     auto packets = encodingStrategy_>encode(frame, sequenceNumber_, timestamp_, ssrc_);  
6  
7     for (auto& packet : packets) {  
8         socket_>sendTo(packet.getPacketVector().data(), packet.getLength(),  
9             clientIP_, clientRTPPort_);  
10        sequenceNumber_ = packet.getSequenceNumber() + 1;  
11    }  
12  
13    timestamp_ += 3600;    // advance timestamp per frame (project convention)  
14    frameTimer_>wait();  
15 }
```

Listing 10: Luồng streaming: đọc frame, encode RTP và gửi UDP

3.3.6 Thuật toán Server Streaming Loop: điều phối chính xác theo FPS

Streaming loop là trái tim của Server, chịu trách nhiệm duy trì tốc độ phát video ổn định. Hình 6 minh họa thuật toán chi tiết:

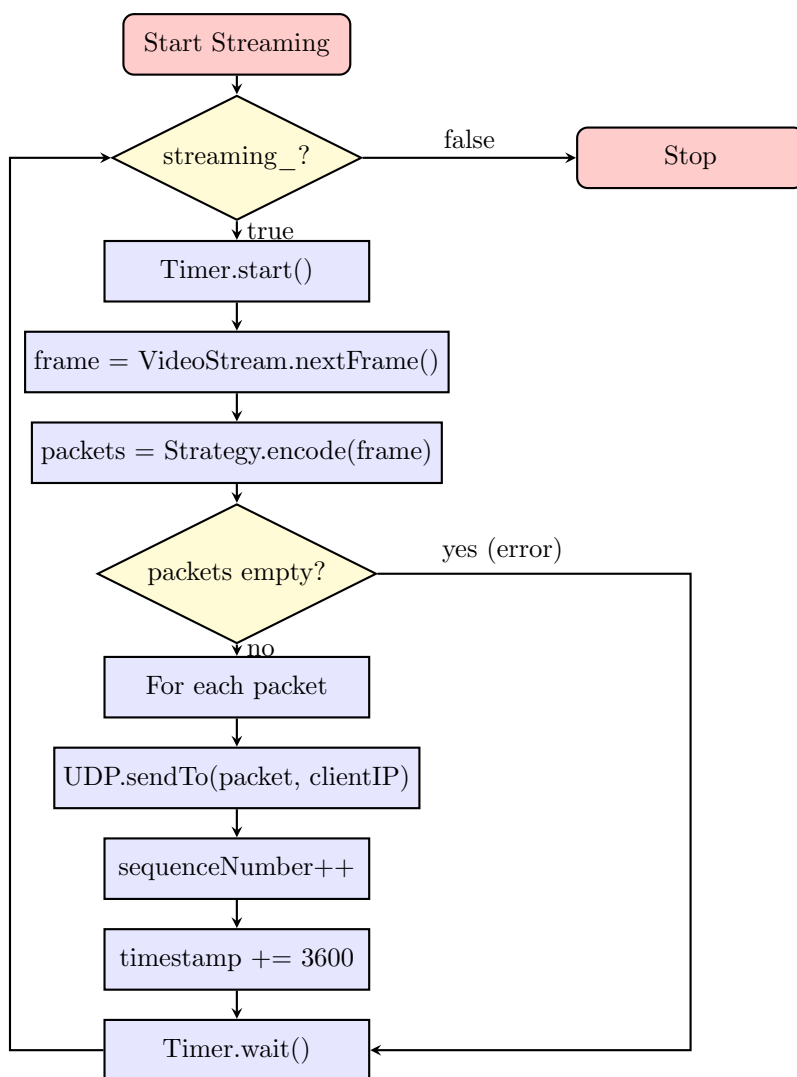


Figure 6: Flowchart thuật toán Server Streaming Loop

Giải thích chi tiết thuật toán:

3.3.6.1 Bước 1 - Kiểm tra cờ streaming_: Đây là atomic boolean cho phép thread chính dừng loop từ bên ngoài (khi nhận PAUSE hoặc TEARDOWN). Việc kiểm tra ở đầu loop đảm bảo thread có thể thoát ngay lập tức.

3.3.6.2 Bước 2 - Timer.start(): Ghi nhận thời điểm bắt đầu xử lý frame. Timer sử dụng `std::chrono::high_resolution_clock` để đo thời gian chính xác. Đây là điểm bắt đầu của "budget thời gian" cho frame này.

3.3.6.3 Bước 3 - Đọc frame từ VideoStream: VideoStream scan file MJPEG để tìm marker SOI/EOI (sẽ giải thích chi tiết ở phần sau). Nếu hết video, `nextFrame()` có thể trả về frame rỗng hoặc loop lại từ đầu (tùy cấu hình).

3.3.6.4 Bước 4 - Encode frame thành RTP packets: Strategy pattern được áp dụng tại đây. `EncodingStrategy` nhận frame (có thể lớn tới vài trăm KB), chia thành nhiều fragments nếu cần, và wrap mỗi fragment vào RTP packet với sequence number tăng dần.

3.3.6.5 Bước 5 - Gửi tất cả packets qua UDP: Vòng lặp đi qua từng packet và gọi `sendTo()` với địa chỉ `clientIP:clientRTPPort`. UDP không đảm bảo thứ tự hay độ tin cậy, nhưng đổi lại có độ trễ thấp. Sequence number trong RTP header sẽ giúp Client sắp xếp lại.

3.3.6.6 Bước 6 - Tăng timestamp: Timestamp trong RTP không phải là wall-clock time, mà là "media time". Dự án sử dụng convention `timestamp += 3600` mỗi frame (tương ứng với 90kHz clock rate của RTP, chia cho 25 FPS). Điều này giúp Client biết khoảng cách thời gian giữa các frame.

3.3.6.7 Bước 7 - Timer.wait(): Đây là bước quan trọng nhất để kiểm soát FPS. Timer tính toán thời gian đã dùng (từ `start()` đến `wait()`) và sleep phần còn lại để đạt interval mục tiêu (ví dụ: 40ms cho 25 FPS). Nếu xử lý frame mất quá lâu ($> 40\text{ms}$), `wait()` sẽ return ngay lập tức mà không sleep, tránh "nợ" thời gian tích lũy.

```
1 void Timer::wait() {  
2     auto now = std::chrono::high_resolution_clock::now();  
3     auto elapsed = std::chrono::duration_cast<std::chrono::milliseconds>(now -  
4         startTime_);  
5     auto remaining = std::chrono::milliseconds(intervalMs_) - elapsed;  
6     if (remaining.count() > 0) {  
7         std::this_thread::sleep_for(remaining);  
8     }
```

```
9 // else: frame processing took too long, skip sleep
10 }
```

Listing 11: `Timer.wait()` chỉ sleep phần thời gian còn lại của interval

Tại sao thiết kế như vậy? Cách tiếp cận "budget-based timing" này đảm bảo:

- FPS ổn định ngay cả khi thời gian xử lý biến động
- Không tích lũy lag khi một frame xử lý chậm (skip sleep thay vì queue up delay)
- Client nhận được luồng video với bitrate dự đoán được, giúp buffer hoạt động hiệu quả

3.3.7 Strategy pattern: SD vs HD và phân mảnh payload

Với video MJPEG, kích thước frame có thể vượt MTU. Để tránh UDP fragmentation ở tầng IP, hệ thống giới hạn `MAX_PAYLOAD_SIZE` (1400 bytes) và tự phân mảnh ở tầng RTP. Strategy pattern được dùng để tách logic encode: `SEncodingStrategy` gửi một packet/1 frame (chỉ phù hợp frame nhỏ), còn `HEncodingStrategy` chia frame thành nhiều packet, dùng marker bit để đánh dấu fragment cuối.

```
1 while (offset < frameData.size()) {
2     size_t chunk = std::min((size_t)RTPPacket::MAX_PAYLOAD_SIZE,
3                             frameData.size() - offset);
4     bool isLast = (offset + chunk >= frameData.size());
5
6     RTPPacket pkt = RTPPacketBuilder()
7         .setSequenceNumber(seqNum++)
8         .setTimestamp(timestamp)
9         .setSSRC(ssrc)
10        .setMarker(isLast ? 1 : 0)
11        .setPayload(&frameData[offset], chunk)
12        .build();
13
14    packets.push_back(pkt);
15    offset += chunk;
16 }
```

Listing 12: HD encoding: chia frame thành nhiều RTP packet và set marker cho fragment cuối

3.3.8 Giải thích chi tiết về RTP Fragmentation: tại sao cần và cách hoạt động

Vấn đề MTU và IP Fragmentation: Mạng Ethernet tiêu chuẩn có MTU (Maximum Transmission Unit) là 1500 bytes. Nếu gói UDP vượt quá MTU, tầng IP sẽ tự động phân mảnh (IP fragmentation). Tuy nhiên, với UDP, nếu *bất kỳ* fragment nào bị mất, toàn bộ gói sẽ bị discard. Đối với video streaming, điều này không chấp nhận được.

Giải pháp Application-Level Fragmentation: Thay vì để IP phân mảnh, hệ thống chủ động chia frame thành nhiều RTP packets nhỏ hơn (mỗi packet ≤ 1400 bytes payload). Khi đó:

- Mỗi RTP packet đi trong một UDP datagram riêng, không bị phân mảnh ở tầng IP
- Nếu một packet bị mất, Client vẫn có thể nhận các packets khác của cùng frame
- Với MJPEG, việc mất một phần frame chỉ gây méo hình một phần, vẫn hiển thị được (graceful degradation)

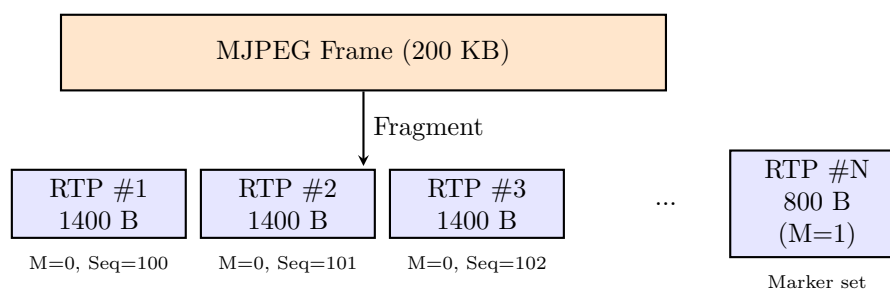


Figure 7: Fragmentation một frame MJPEG thành nhiều RTP packets

Vai trò của Marker Bit: Bit M (Marker) trong RTP header được set = 1 *chỉ* ở packet cuối cùng của frame. Client sử dụng marker để biết khi nào đã nhận đủ fragments và có thể reassemble. Nếu không có marker, Client sẽ không biết khi nào frame kết thúc, đặc biệt khi các frames có kích thước khác nhau.

3.3.9 VideoStream: thuật toán trích xuất frame MJPEG bằng marker JPEG

File `.Mjpeg` trong project là chuỗi các ảnh JPEG nối tiếp. Để trích xuất frame, `VideoStream` đọc file vào buffer và tìm cặp marker JPEG: SOI `0xFF 0xD8` (Start Of Image) và EOI `0xFF 0xD9` (End Of Image). Khi tìm được EOI, lớp này cắt đoạn byte tương ứng và trả về như một frame hoàn chỉnh.

```
1 while (file_.read((char*)&byte, 1)) {
```

```
2    buffer_.push_back(byte);
3
4    if (buffer_.size() >= 2) {
5        uint8_t b1 = buffer_[buffer_.size()-2];
6        uint8_t b2 = buffer_[buffer_.size()-1];
7
8        if (!inFrame && b1 == 0xFF && b2 == 0xD8) {
9            inFrame = true; frameStart = buffer_.size() - 2;
10        }
11        if (inFrame && b1 == 0xFF && b2 == 0xD9) {
12            size_t frameEnd = buffer_.size();
13            std::vector<uint8_t> frame(buffer_.begin()+frameStart,
14            buffer_.begin()+frameEnd);
15            buffer_.erase(buffer_.begin(), buffer_.begin()+frameEnd);
16            return frame;
17        }
18    }
```

Listing 13: Quét marker JPEG SOI/EOI để cắt frame trong `VideoStream::nextFrame()`

Thuật toán này dựa trên cấu trúc của định dạng JPEG: mỗi ảnh JPEG bắt đầu bằng marker SOI (Start Of Image: 0xFF 0xD8) và kết thúc bằng EOI (End Of Image: 0xFF 0xD9). File `.Mjpeg` chỉ đơn giản là nhiều ảnh JPEG nối tiếp nhau mà không có delimiter hay header đặc biệt.

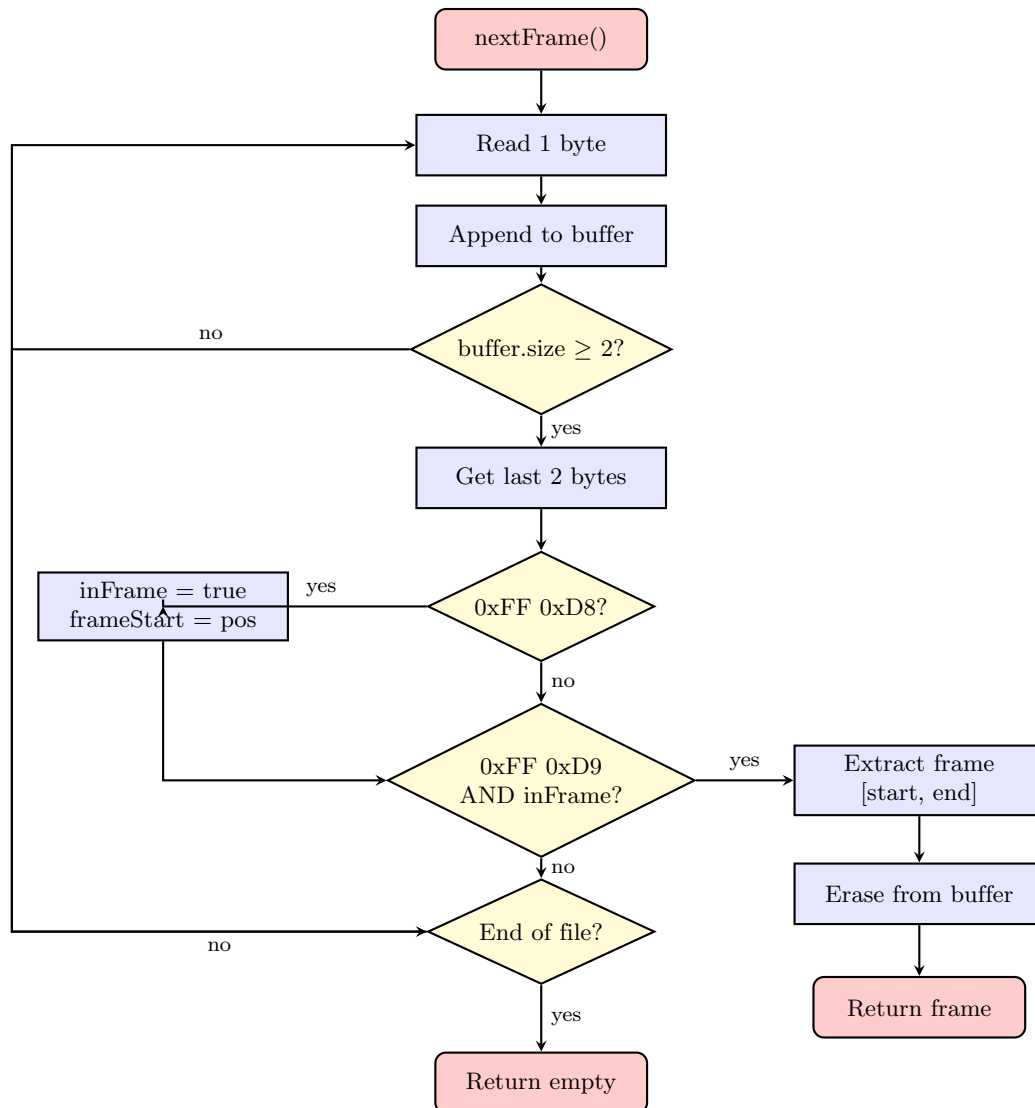


Figure 8: Flowchart of MJPEG frame extraction algorithm

Giải thích chi tiết thuật toán:

3.3.9.1 Tại sao cần buffer tích lũy? Không thể đọc từng ảnh JPEG một cách trực tiếp vì không biết trước kích thước của mỗi ảnh. Thuật toán phải scan tuần tự để tìm marker. Buffer (`std::vector<uint8_t>`) tích lũy bytes cho đến khi phát hiện được một frame hoàn chỉnh.

3.3.9.2 Phát hiện SOI: Khi gặp `0xFF 0xD8`, đánh dấu `inFrame = true` và ghi nhận vị trí `frameStart`. Điều này cho biết "từ đây trở đi là dữ liệu của một frame".

3.3.9.3 Phát hiện EOI: Khi gặp 0xFF 0xD9 VÀ `inFrame == true`, ta biết frame đã hoàn chỉnh. Lúc này extract đoạn `[frameStart, currentPos]` từ buffer, xóa phần đã extract để tiết kiệm bộ nhớ, và return frame.

3.3.9.4 Xử lý khi hết file: Nếu đọc hết file mà chưa tìm thấy EOI (có thể do file corrupt hoặc frame cuối bị cắt), trả về frame rỗng. Điều này khiến streaming loop có thể phát hiện end-of-stream.

3.3.9.5 Tối ưu hóa bộ nhớ: Sau khi extract frame, thuật toán xóa phần đã dùng khỏi buffer bằng `buffer_.erase()`. Điều này quan trọng vì video file có thể có hàng nghìn frames; nếu không xóa, buffer sẽ phình to không kiểm soát.

3.4 Client: RTSP điều khiển, nhận RTP, ghép frame và buffer/cache

3.4.1 RTSPClient: phiên điều khiển và cơ chế “restart”

`RTSPClient` chịu trách nhiệm tạo kết nối TCP đến server, gửi các request RTSP (SETUP/-PLAY/PAUSE/TEARDOWN), và parse response để lấy `Session` ID. Với yêu cầu “replay” sau khi hết video, client hỗ trợ cơ chế restart bằng header `Restart: true` trong PLAY; server kiểm tra header này để reset `VideoStream` về đầu file.

```
1 std::map<std::string, std::string> headers;  
2 headers["Session"] = sessionId_;  
3 if (restart) headers["Restart"] = "true";  
4  
5 std::string request = RTSPMessage::buildRequest("PLAY", rtspUri_, cseq_, headers);  
6 socket_->send((const uint8_t*)request.c_str(), request.size());
```

Listing 14: Client gửi PLAY kèm header Restart khi cần phát lại

3.4.2 RTPReceiver: nhận UDP và thống kê mất gói

`RTPReceiver` chạy background thread `receiveLoop()` để nhận packet và chuyển cho `FrameReassembler`. Đồng thời, receiver cập nhật thống kê loss dựa trên chênh lệch sequence; phần này dựa trực tiếp trên `RTPPacket::sequenceDifference()` để xử lý wrap-around.

```
1 int32_t diff = RTPPacket::sequenceDifference(currentSeq, lastSequenceNumber_);
```

```
2 if (diff > 1) {  
3     packetLost_ += static_cast<uint64_t>(diff - 1);  
4 }  
5 lastSequenceNumber_ = currentSeq;
```

Listing 15: Tính packet loss dựa trên chênh lệch sequence

3.4.3 Giải thích chi tiết RTP Sequence Number Wrap-around

Trước khi đi vào thuật toán reassembly, cần hiểu rõ vấn đề wrap-around của sequence number. RTP sử dụng sequence number 16-bit, có nghĩa là sau 65535 packets, nó sẽ quay về 0. Điều này gây ra vấn đề khi so sánh thứ tự:

Đoạn mã dưới đây minh họa phép so sánh trực tiếp bị sai khi sequence wrap-around.

```
1 uint16_t seq1 = 65535; // sent earlier  
2 uint16_t seq2 = 0;     // sent later after wrap  
3 if (seq1 < seq2) { /* ... */ }
```

Listing 16: Wrap-around: so sánh trực tiếp bị sai

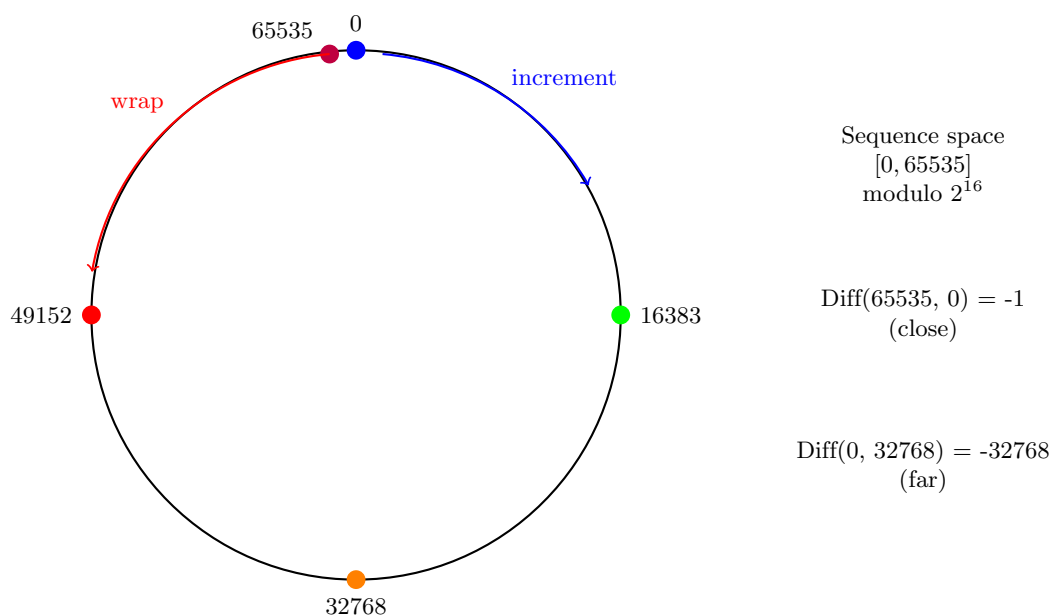


Figure 9: Không gian sequence number 16-bit và wrap-around

Công thức tính khoảng cách sequence đúng:

```
1 int32_t RTPPacket::sequenceDifference(uint16_t seq1, uint16_t seq2) {
2     int32_t diff = (int32_t)seq1 - (int32_t)seq2;
3
4     // If diff > 32768, seq2 has wrapped past 0 and is greater than seq1
5     // Example: seq1=100, seq2=65000 -> diff=100-65000=-64900
6     // Actually seq2 is older, distance = -64900 + 65536 = 636
7     if (diff > 32768) diff -= 65536;
8
9     // If diff < -32768, seq1 has wrapped past 0 and is greater than seq2
10    // Example: seq1=65000, seq2=100 -> diff=65000-100=64900
11    // Actually seq1 is older, distance = 64900 - 65536 = -636
12    if (diff < -32768) diff += 65536;
13
14    return diff;
15 }
```

Listing 17: sequenceDifference function handles wrap-around correctly

Ứng dụng trong phát hiện packet loss:

```
1 int32_t diff = RTPPacket::sequenceDifference(currentSeq, lastSeq);
2 if (diff > 1) {
3     // Example: lastSeq=65534, currentSeq=2
4     // diff = sequenceDifference(2, 65534)
5     //      = 2 - 65534 = -65532
6     //      = -65532 + 65536 = 4
7     // Lost packets = 4 - 1 = 3 (packets 65535, 0, 1)
8     packetLost_ += (diff - 1);
9 }
```

Listing 18: Calculate packet loss correctly with wrap-around

3.4.4 FrameReassembler: thuật toán ghép fragment phức tạp với xử lý out-of-order

Đối với HD mode, một frame có thể gồm nhiều RTP packet. **FrameReassembler** gom các fragment theo **timestamp**, sắp thứ tự theo **sequenceNumber**, và xác định điểm kết thúc frame

dựa trên marker bit. Khi frame hoàn chỉnh, reassembler nối payload và push vào `FrameBuffer`. Để tránh leak khi mất marker (mất gói), module cũng hỗ trợ giới hạn bộ nhớ cho map và chiến lược “flush” khi đổi timestamp.

```
1 if (packet.getMarker() == 1) {
2     auto& fragments = frameFragments_[timestamp];
3     std::sort(fragments.begin(), fragments.end(),
4         [](auto& a, auto& b){ return a.getSequenceNumber() <
5             b.getSequenceNumber(); });
6
7     std::vector<uint8_t> frameData;
8     for (auto& frag : fragments) {
9         const auto& payload = frag.getPayload();
10        frameData.insert(frameData.end(), payload.begin(), payload.end());
11    }
12
13    frameBuffer_->push(frameData);
14    frameFragments_.erase(timestamp);
15 }
```

Listing 19: Ghép frame khi gặp marker bit và đẩy vào buffer

Thuật toán Reassembly phải xử lý nhiều tình huống phức tạp: packets đến không theo thứ tự, packets bị mất, và nhiều frames đang được reassemble đồng thời. Hình 10 minh họa logic đầy đủ:

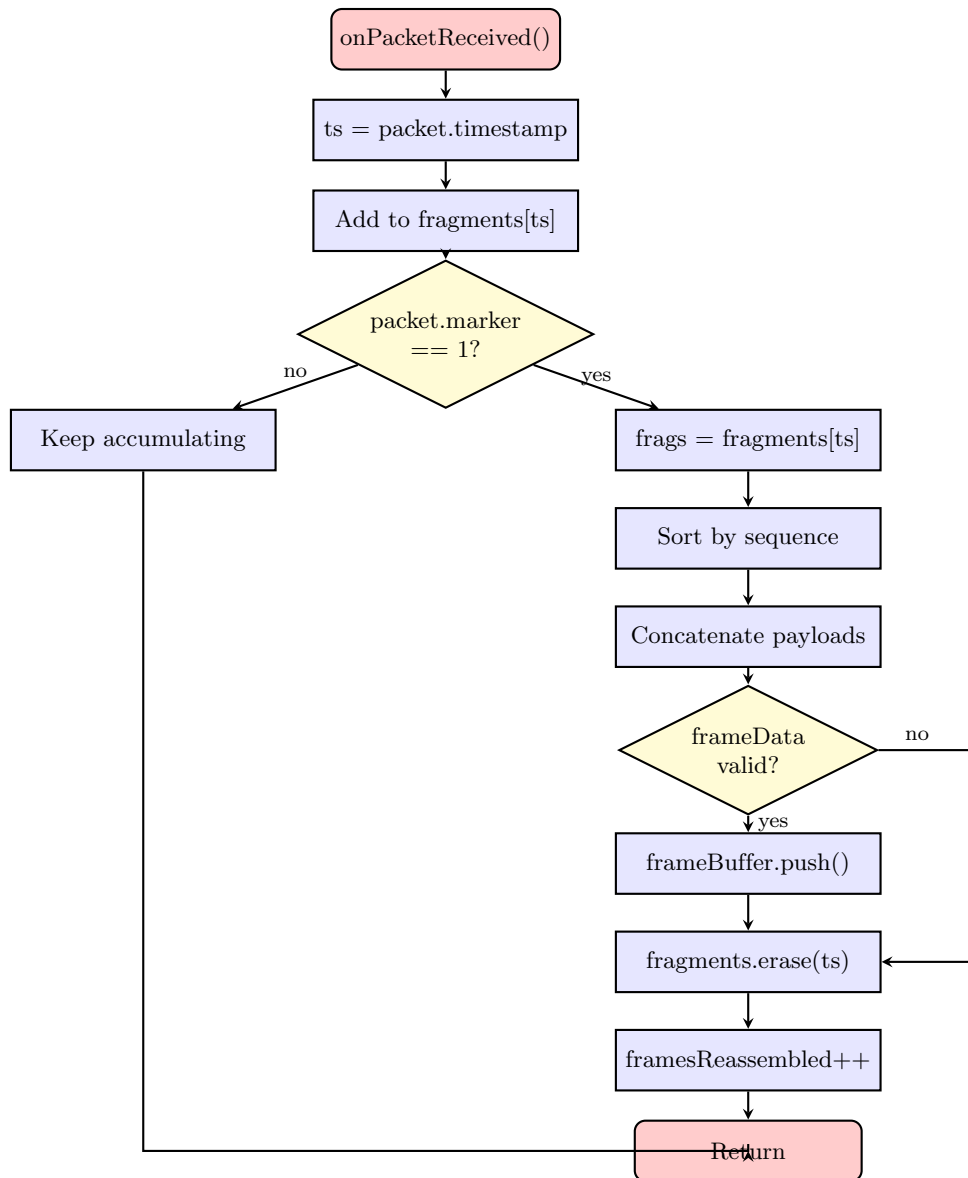


Figure 10: Flowchart of Frame Reassembly algorithm

Giải thích chi tiết các bước quan trọng:

3.4.4.1 Bước 1 - Nhóm theo timestamp: Tất cả fragments của cùng một frame có cùng RTP timestamp. Reassembler dùng cấu trúc `std::map` với key là `uint32_t` timestamp và value là `std::vector<RTPPacket>` để nhóm. Mỗi entry trong map chứa danh sách packets chưa sắp xếp của một frame.

3.4.4.2 Bước 2 - Kiểm tra marker bit: Chỉ khi nhận được packet có `marker=1`, reassembler mới biết "frame này đã hoàn chỉnh (ít nhất về mặt gửi)". Trước đó, các fragments chỉ được tích lũy mà không xử lý.

3.4.4.3 Bước 3 - Sắp xếp theo sequence number: Do UDP không đảm bảo thứ tự, các fragments có thể đến lung tung. Thuật toán sort theo sequence để ghép đúng thứ tự. Đây là lúc hàm `sequenceDifference()` quan trọng để so sánh chính xác.

```
1 std::sort(fragments.begin(), fragments.end(),
2         [](const RTPPacket& a, const RTPPacket& b) {
3             // Cannot use a.seq < b.seq due to wrap-around
4             // Use sequenceDifference for correct comparison
5             return RTPPacket::sequenceDifference(a.getSequenceNumber(),
6                                                  b.getSequenceNumber()) < 0;
7         });
```

Listing 20: Sort fragments handling wrap-around

3.4.4.4 Bước 4 - Concatenate payloads: Sau khi sort, nối tất cả payloads thành một `std::vector<uint8_t>` duy nhất. Đây chính là dữ liệu JPEG hoàn chỉnh.

3.4.4.5 Bước 5 - Validation: Kiểm tra `frameData` không rỗng và có thể decode được (optional: có thể check JPEG header `0xFF 0xD8`). Nếu invalid (có thể do mất quá nhiều fragments), discard frame.

3.4.4.6 Bước 6 - Push vào FrameBuffer và cleanup: Frame hoàn chỉnh được đẩy vào buffer thread-safe. Sau đó xóa entry trong `fragments` map để giải phóng bộ nhớ. Điều này quan trọng vì nếu không xóa, map sẽ phình to với các frames cũ.

Xử lý edge cases:

- **Mất marker packet:** Nếu packet cuối (có marker) bị mất, frame sẽ không bao giờ được reassemble. Để tránh leak memory, hệ thống có thể implement timeout: nếu một timestamp không nhận thêm packet sau X giây, force flush hoặc discard.
- **Packets đến muộn:** Nếu packet có timestamp cũ đến sau khi frame đó đã được reassemble (entry đã bị erase), packet sẽ bị ignore. Đây là trade-off giữa latency và completeness.

- **Out-of-order severe:** Nếu packet có sequence number quá xa so với các packets khác của cùng timestamp, có thể là lỗi. Code có thể log warning để debug.

3.4.5 FrameBuffer: buffer như một “video cache” thread-safe

FrameBuffer là lớp trung gian quan trọng nhất để “hấp thụ jitter” mạng. Nó được hiện thực như một hàng đợi thread-safe, với `push()` từ thread nhận RTP và `tryPop()` từ UI thread. `std::mutex` và `std::condition_variable` được dùng để đồng bộ; `close()` đảm bảo thread consumer không bị kẹt khi shutdown.

```
1 void FrameBuffer::push(const std::vector<uint8_t>& frame) {
2     std::unique_lock<std::mutex> lock(mtx_);
3     if (closed_) return;
4     buffer_.push_back(frame);
5     cv_.notify_one();
6 }
7
8 bool FrameBuffer::tryPop(std::vector<uint8_t>& frame) {
9     std::unique_lock<std::mutex> lock(mtx_);
10    if (buffer_.empty()) return false;
11    frame = std::move(buffer_.front());
12    buffer_.pop_front();
13    return true;
14 }
```

Listing 21: FrameBuffer đồng bộ bằng mutex + condition variable

3.4.6 FrameBuffer: vai trò như video cache và cơ chế Producer-Consumer

Tại sao cần FrameBuffer? Nếu UI render frame ngay khi nhận được từ mạng, video sẽ giật liên tục do jitter mạng. FrameBuffer đóng vai trò như một "bể chứa" (reservoir) để:

- Tách nhịp nhận mạng (bursty, không đều) khỏi nhịp hiển thị (đều đặn 25 FPS)
- Hấp thụ jitter: khi mạng chậm, buffer cung cấp frames từ dự trữ
- Cho phép prebuffering để tránh underrun ngay từ đầu

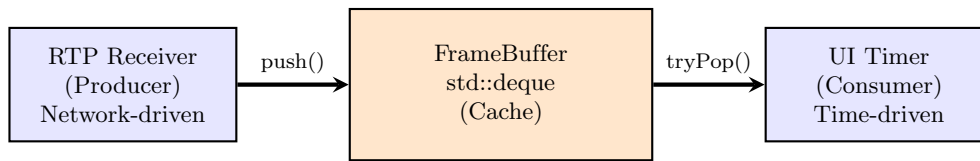


Figure 11: Mô hình Producer-Consumer của FrameBuffer

Thread-safety implementation: Do producer (RTP thread) và consumer (UI thread) chạy song song, cần đồng bộ hóa. Code sử dụng:

- `std::mutex` để bảo vệ `std::deque`
- `std::condition_variable` để consumer có thể đợi hiệu quả thay vì busy-wait
- Cờ `closed_` để xử lý shutdown gracefully

3.4.7 ClientUI (Qt): prebuffer, adaptive playback và cơ chế chống jitter

Ở phía UI, state machine được đơn giản hóa thành `INIT/READY/PLAYING`. Khi `PLAY`, UI không render ngay lập tức mà đợi buffer đạt ngưỡng `PREBUFFER_FRAMES`; đây là cơ chế prebuffer nhằm tạo “độ trễ chủ động” khoảng vài giây để chống đứt quãng. Đồng thời, UI điều chỉnh interval của `frameTimer_` theo kích thước buffer để tránh tiêu thụ hết buffer khi mạng chậm.

```
1 if (!prebufferReady_) {
2     size_t bufferSize = frameBuffer_>size();
3     if (bufferSize >= PREBUFFER_FRAMES) {
4         prebufferReady_ = true;
5     } else {
6         statusLabel_>setText(QString("Buffering... %1/%2 frames")
7                                 .arg(bufferSize).arg(PREBUFFER_FRAMES));
8         return;
9     }
10 }
```

Listing 22: Prebuffer: chỉ phát khi buffer đủ `PREBUFFER_FRAMES`

```
1 if (currentBufferSize >= 20)    frameTimer_>setInterval(38);
2 else if (currentBufferSize >= 10) frameTimer_>setInterval(40);
3 else if (currentBufferSize >= 5)  frameTimer_>setInterval(45);
```

```
4 else frameTimer_ -> setInterval(50);
```

Listing 23: Adaptive timer: tăng interval khi buffer thấp để giảm giật

Giải thích chi tiết cơ chế Prebuffering:

Prebuffering là kỹ thuật "trả độ trễ để lấy ổn định". Thay vì phát ngay khi nhận được frame đầu tiên, Client đợi cho đến khi tích lũy đủ PREBUFFER_FRAMES (ví dụ: 50 frames $\approx$ 2 giây ở 25 FPS). Lý do:

- **Khởi động ổn định:** Vài frame đầu có thể đến chậm do TCP slow start, DNS lookup, etc. Nếu phát ngay, buffer sẽ cạn trong vài giây đầu.
- **Chống jitter đầu phát:** Tạo "vùng đệm dự phòng" để hấp thụ biến động mạng trong giai đoạn đầu.
- **User experience:** Người dùng thích chờ 2 giây để load rồi xem mượt hơn là xem giật liên tục.

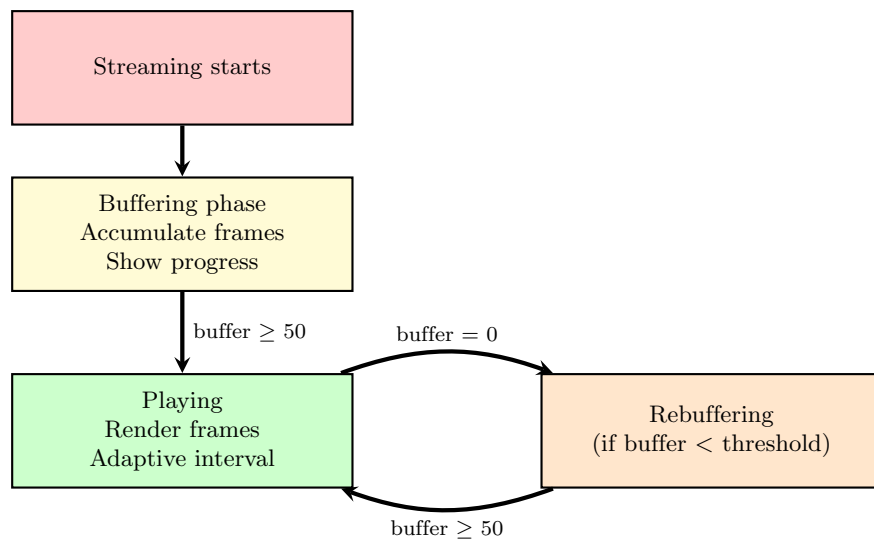


Figure 12: State flow of prebuffering mechanism

Giải thích chi tiết cơ chế Adaptive Timer:

Adaptive timer điều chỉnh tốc độ render dựa trên mức buffer hiện tại. Ý tưởng: khi buffer thấp, render chậm lại (tăng interval) để cho mạng kịp "bắt kịp". Khi buffer cao, render nhanh hơn để tránh lag tích lũy.

Table 6: Chiến lược adaptive interval dựa trên buffer size

Buffer Size	Interval (ms)	Lý do
≥ 20 frames	38	Buffer dồi dào, render nhanh (26.3 FPS) để tiêu thụ buffer
≥ 10 frames	40	Mức bình thường, đúng 25 FPS mục tiêu
≥ 5 frames	45	Buffer thấp, chậm lại (22.2 FPS) để tránh cạn
< 5 frames	50	Nguy cơ underrun, render rất chậm (20 FPS)

Tại sao không giữ cố định 25 FPS? Nếu render cố định 40ms/frame nhưng mạng chỉ cung cấp trung bình 45ms/frame (do bandwidth thấp), buffer sẽ cạn dần và video sẽ giật. Adaptive timer "nhượng bộ" bằng cách giảm FPS xuống 20-22 để buffer có thể hồi phục.

Tại sao không tăng interval lên cao hơn (ví dụ 100ms)? Vì điều đó làm video quá chậm (10 FPS) và không còn watchable. 20 FPS là ngưỡng thấp nhất mà con người vẫn cảm nhận được chuyển động tương đối mượt.

```
1 void ClientUI::onRenderFrame() {
2     size_t bufferSize = frameBuffer_>size();
3
4     // If buffer is completely empty, trigger rebuffering
5     if (bufferSize == 0) {
6         prebufferReady_ = false; // return to buffering state
7         return;
8     }
9
10    // Adjust interval based on buffer level
11    if (bufferSize >= 20)    frameTimer_>setInterval(38); // 26.3 FPS
12    else if (bufferSize >= 10) frameTimer_>setInterval(40); // 25 FPS
13    else if (bufferSize >= 5)  frameTimer_>setInterval(45); // 22.2 FPS
14    else                      frameTimer_>setInterval(50); // 20 FPS
15
16    // Render frame
17    std::vector<uint8_t> frame;
18    if (frameBuffer_>tryPop(frame)) {
19        displayFrame(frame);
20        currentFrame_++;
```

```
21 }  
22 }
```

Listing 24: Adaptive interval logic in UI timer callback

Kết quả của cơ chế này: Video có thể tự động thích nghi với điều kiện mạng:

- Mạng tốt (> 25 FPS): buffer cao, render 26 FPS, smooth
- Mạng trung bình ($= 25$ FPS): buffer ổn định, render 25 FPS
- Mạng chậm (< 25 FPS): buffer giảm dần, render chậm xuống 20-22 FPS, vẫn xem được
- Mạng rất chậm: buffer cạn, trigger rebuffering, chờ buffer đầy rồi tiếp tục

Timeline slider được mở rộng thành `BufferedSlider` để hiển thị vùng đã buffer (“buffered position”), tính bằng `currentFrame + bufferSize`. Phần vẽ bổ sung được hiện thực trong `paintEvent()`.

```
1 int bufferedPos = (bufferedValue_ - minimum()) * groove.width() / total;  
2 if (bufferedPos > currentPos) {  
3     QRect bufferedRect = groove;  
4     bufferedRect.setLeft(groove.left() + currentPos);  
5     bufferedRect.setRight(groove.left() + bufferedPos);  
6     painter.fillRect(bufferedRect, QColor(180, 180, 180, 200));  
7 }
```

Listing 25: Vẽ vùng buffered trên timeline slider

3.5 Ghi nhận triển khai: logging, cấu hình và khả năng mở rộng

Hệ thống sử dụng `Logger` (Singleton) để log đồng thời ra console và file, đảm bảo thread-safe bằng `std::mutex`. Việc cấu hình tách riêng ở `Config` (SingletonWithInit) cho phép thay đổi tham số runtime (port, video file, log level) mà không cần rebuild.

```
1 void Logger::log(LogLevel level, const std::string& message) {  
2     std::lock_guard<std::mutex> lock(mutex_);  
3     if (!initialized_ || level < minLevel_) return;  
4     std::cout << "[" << getCurrentTime() << " ] [" << levelToString(level) << " ] "  
5         << message << std::endl;
```

```
6  if (logFile_.is_open()) logFile_ << ...;  
7 }
```

Listing 26: Logger Singleton: log thread-safe và lọc theo mức

Về khả năng mở rộng, các điểm đã “cắm sẵn” trong thiết kế gồm: Strategy cho encoding (thêm H.264/H.265 packetization), Builder cho các message phức tạp hơn, và RTSP state machine tách handler để bổ sung DESCRIBE/SET_PARAMETER/RANGE trong tương lai mà không làm vỡ luồng chính.

4 Results

Phần này tổng hợp kết quả kiểm thử và xác nhận tính đúng đắn của các khối logic cốt lõi (RTSP message handling, Timer kiểm soát nhịp), đồng thời mô tả các hành vi runtime có thể quan sát được từ client UI và các thống kê có sẵn trong mã nguồn.

4.1 Kết quả kiểm thử đơn vị (Unit Tests)

Dự án cung cấp bộ kiểm thử trong `common/tests` cho hai thành phần có rủi ro lỗi cao: (1) xây dựng và phân tích RTSP message, (2) Timer điều phối thời gian. Các test được viết theo phong cách “lightweight framework” (assert + reporter) và có thể chạy độc lập với Qt.

Table 7: Tổng hợp kết quả unit test ở tầng Common

Test Suite	Số test	Kết quả
<code>test_rtsp_message</code>	15	15 passed, 0 failed
<code>test_timer</code>	13	13 passed, 0 failed

Kết quả chạy test RTSPMessage thể hiện cả các kịch bản cơ bản lẫn round-trip (build rồi parse lại). Đoạn log dưới đây minh họa phần tổng kết của suite (trích nguyên văn từ output).

```
1 Total: 15 tests
2 Passed: 15
3 Failed: 0
```

Listing 27: Output tổng kết của `test_rtsp_message`

Tương tự, Timer được kiểm tra với nhiều interval (25 FPS, 30 FPS, 60 FPS), kiểm tra drift (không sleep khi “work time” vượt interval), và stress test. Đoạn log tổng kết:

```
1 Total: 13 tests
2 Passed: 13
3 Failed: 0
```

Listing 28: Output tổng kết của `test_timer`

4.2 Chỉ số runtime sẵn có ở Client

Ngoài unit test, Client đã tích hợp các thống kê runtime cho RTP: số packet nhận, số packet ước lượng mất, tổng bytes, và phần trăm loss. Loss được tính bằng chênh lệch sequence; do đó thống kê phản ánh mất gói “theo thứ tự” (out-of-order có thể bị tính như lost nếu đến quá muộn hoặc bị bỏ qua ở Reassembler tùy kịch bản).

```
1 double RTPReceiver::getPacketLossPercentage() const {  
2     uint64_t total = packetReceived_ + packetLost_;  
3     if (total == 0) return 0.0;  
4     return (static_cast<double>(packetLost_) * 100.0) / static_cast<double>(total);  
5 }
```

Listing 29: Công thức phần trăm mất gói RTP

4.3 Hành vi buffer và prebuffer quan sát từ UI

UI triển khai hai cơ chế nhằm tối ưu trải nghiệm: (1) prebuffer trước khi phát, (2) điều chỉnh nhịp render theo mức buffer. Prebuffer được kích hoạt khi bắt đầu PLAY hoặc khi xảy ra “rebuffering” do buffer cạn; điều này làm tăng độ ổn định, đổi lại là tăng độ trễ đầu phát.

Table 8: Các tham số buffer quan trọng và ý nghĩa vận hành

Tham số	Vị trí	Ý nghĩa
PREBUFFER_FRAMES	ClientUI	Số frame cần tích lũy trước khi bắt đầu render
LOW_BUFFER_THRESHOLD	ClientUI	Ngưỡng cảnh báo buffer thấp và tăng interval render
MAX_PAYLOAD_SIZE	RTPPacket	Giới hạn payload/packet để tránh IP fragmentation

```
1 statusLabel_->setText(QString("Buffering... %1/%2 frames (%3%)")  
2     .arg(bufferSize)  
3     .arg(PREBUFFER_FRAMES)  
4     .arg(percentage, 0, 'f', 1));
```

Listing 30: UI chuyển sang trạng thái “Buffering” khi buffer chưa đủ

4.4 Hướng dẫn triển khai và tái lập kết quả

4.4.1 Thiết lập dự án thông qua `setup.py`

Để đơn giản hóa quy trình triển khai trên các nền tảng khác nhau, dự án cung cấp script tự động hóa `setup.py` thực hiện toàn bộ các bước từ kiểm tra phụ thuộc đến biên dịch mã nguồn.

4.4.1.1 Quy trình tự động Script `setup.py` được thiết kế để hoạt động trên Linux, Windows (MSYS2), và macOS. Khi chạy, script sẽ:

Phát hiện hệ điều hành : Sử dụng `platform.system()` để xác định môi trường thực thi
Kiểm tra Qt6: Tìm kiếm Qt6 qua các phương pháp:

- Kiểm tra `qmake6` trong PATH
- Thực hiện CMake `find_package`
- Tìm kiếm trong đường dẫn cài đặt phổ biến (MSYS2 MinGW64, Homebrew, v.v.)

Hướng dẫn cài đặt Qt6 : Nếu không tìm thấy, hiển thị hướng dẫn chi tiết theo từng nền tảng

Cấu hình CMake : Tự động thiết lập `CMAKE_PREFIX_PATH` và generator phù hợp (Make/Ninja)

Biên dịch song song : Sử dụng `make -j` hoặc `ninja` với tất cả các lõi CPU

Hiển thị hướng dẫn sử dụng : In ra cú pháp chạy server/client với ví dụ cụ thể

Cách sử dụng

```
1 # Linux/macOS
2 python3 setup.py
3
4 # Windows (CMD/PowerShell)
5 python setup.py
```

Listing 31: Chạy script setup tự động

4.4.1.2 Xử lý lỗi phổ biến Script tích hợp xử lý các tình huống lỗi thường gặp:

Lỗi	Xử lý tự động
Qt6 không tìm thấy	Hiển thị hướng dẫn cài đặt chi tiết theo nền tảng (MSYS2 cho Windows, apt/pacman cho Linux, Homebrew cho macOS)
CMake chưa cài đặt	Dừng và yêu cầu cài CMake với link tải
Không có quyền ghi thư mục	Gợi ý thay đổi quyền hoặc clone vào thư mục khác
Build thất bại	In đầy đủ stderr và gợi ý kiểm tra compiler/dependencies

4.4.2 Chạy ứng dụng sau khi build

Sau khi script hoàn tất, các file thực thi nằm trong thư mục `build/bin/`. Cú pháp dòng lệnh được thiết kế đơn giản và trực quan.

4.4.2.1 Khởi động Server

```
1 cd build/bin
2 ./server <port>
3
4 # Example: Run on port 8554
5 ./server 8554
```

Listing 32: Chạy server với port chỉ định

Server sẽ lắng nghe kết nối RTSP trên port được chỉ định (mặc định 8554 theo chuẩn RTSP) và tự động bind UDP socket cho RTP.

4.4.2.2 Khởi động Client

```
1 cd build/bin
2 ./client <server_ip> <port> <video_file>
3
4 # Example: Connect to local server
5 ./client 127.0.0.1 8554 movie.Mjpeg
6
7 # Example: Connect to remote server
8 ./client 192.168.1.100 8554 video.Mjpeg
```

Listing 33: Chạy client với các tham số kết nối

Client sẽ hiển thị giao diện Qt6 với các nút điều khiển RTSP (SETUP, PLAY, PAUSE, TEARDOWN) và thanh timeline để tua video.

4.4.3 Thu thập dữ liệu và log

Trong quá trình chạy, hệ thống ghi log chi tiết vào:

- `server.log`: Các sự kiện phía server (accept connection, session state, streaming loop)
- `client.log`: Các sự kiện phía client (RTSP response, RTP statistics, buffer state)

Client UI cũng hiển thị thống kê real-time: số packet nhận, số packet mất, tỷ lệ loss, buffer size, prebuffer progress, cho phép quan sát hành vi hệ thống mà không cần đọc log file.

5 Giao diện của Client

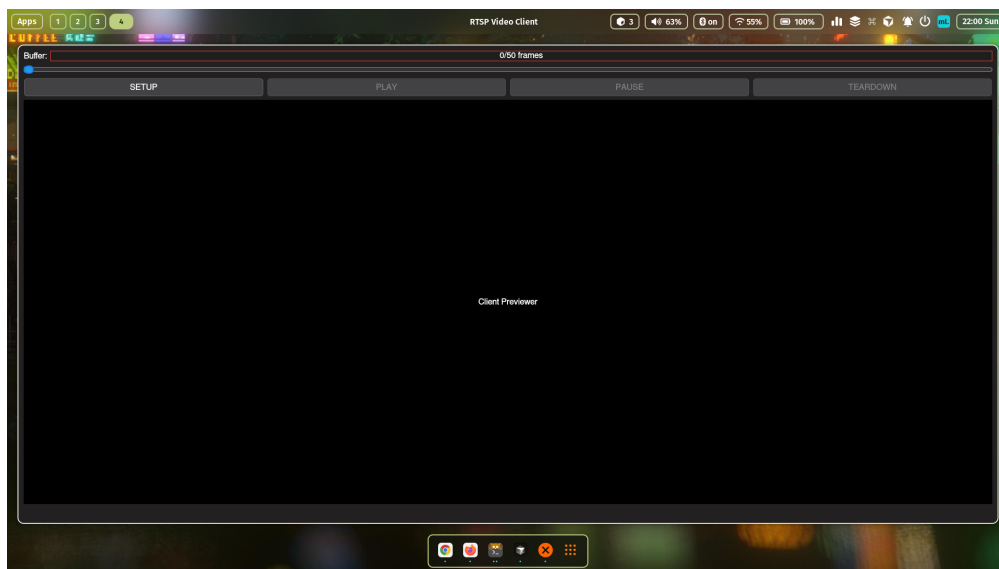


Figure 13: Giao diện của Client

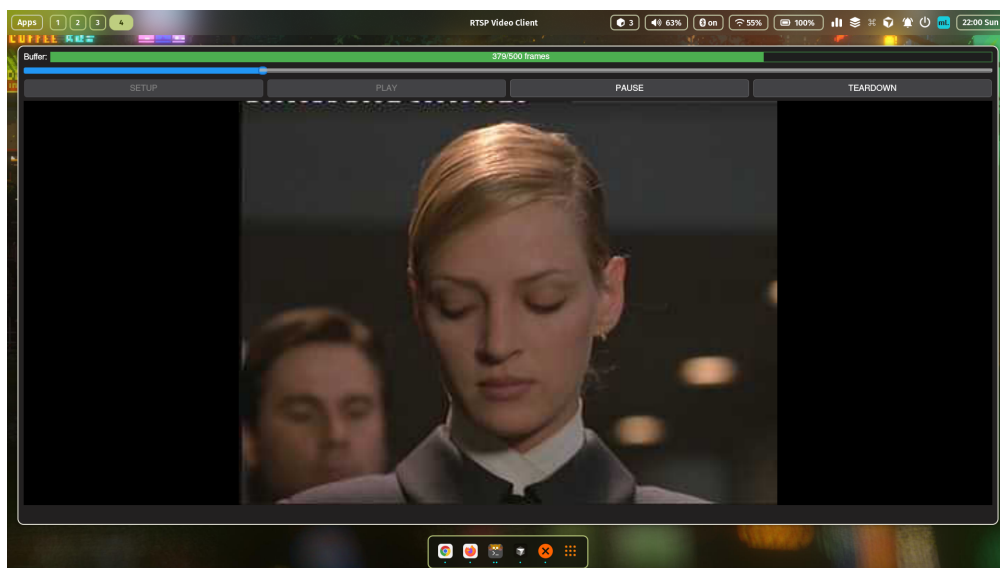


Figure 14: Giao diện của Client

6 Video Demo

Dự án này được xây dựng có thể phù hợp với đa nền tảng khác nhau và đây là các video demo dự án từ đầu

Windows : Xem tại <https://youtu.be/yzEE6nu5VPk>

Linux/MacOS : Xem tại <https://youtu.be/v06-jbP4yDU>

7 Conclusion

Hệ thống đã hiện thực đầy đủ một pipeline streaming video MJPEG theo mô hình RTSP (control plane, TCP) và RTP (data plane, UDP), với cấu trúc module hóa rõ ràng (Common/Server/Client), cơ chế đa luồng để phục vụ đồng thời nhiều client, và cơ chế buffer/prebuffer ở phía client để ổn định playback khi mạng jitter.

7.1 Tổng kết đóng góp kỹ thuật theo mã nguồn

Đóng góp cốt lõi của project không chỉ nằm ở việc “chạy được streaming”, mà ở các quyết định hiện thực giúp hệ thống dễ bảo trì và mở rộng: chuẩn hóa RTSP ở `RTSPMessage`, chuẩn hóa networking ở `Socket`, áp dụng Builder cho RTP packet để kiểm soát tính hợp lệ, Strategy cho encode để tách SD/HD và phân mảnh.

```
1 RTSPClient::connect() -> Socket(TCP)::connect(serverIP, serverPort);
2 RTPReceiver::start() -> Socket(UDP)::bind("0.0.0.0", clientRTPPort);
3 ServerWorker::handleSetup() -> parse Transport header (client_port=...)
4 ServerWorker::streamingLoop() -> Socket(UDP)::sendTo(..., clientIP, clientRTPPort);
```

Listing 34: Tách biệt control plane và data plane: RTSP qua TCP, RTP qua UDP

Ngoài ra, lớp `FrameBuffer` có thể xem là một dạng “video cache” in-memory: nó lưu các frame đã ghép, tách biệt nhịp nhận (network-driven) khỏi nhịp phát (UI-driven). Đây là nền tảng để hiện thực prebuffer và các chính sách điều chỉnh nhịp phát theo mức buffer.

```
1 /* Producer thread */
2 frameReassembler_ -> frameBuffer() -> push(frameData);
3
4 /* Consumer (UI timer) */
5 std::vector<uint8_t> frame;
6 if (frameBuffer_ -> tryPop(frame)) { render(frame); }
```

Listing 35: Buffer như cache: producer (RTP) và consumer (UI) tách nhịp

7.2 Hạn chế hiện tại và đề xuất cải tiến dựa trên bằng chứng mã nguồn

Một số điểm trong code hiện tại có thể ảnh hưởng độ ổn định hoặc độ chính xác timing; các đề xuất dưới đây được nêu dựa trực tiếp trên các đoạn mã hiện có.

7.2.0.1 (1) Tham số Timer ở Server chưa khớp mục tiêu 25 FPS. Trong `ServerWorker` có chú thích “25 FPS (40ms)” nhưng interval đang được khởi tạo là 10ms, dẫn tới nhịp gửi nhanh hơn dự kiến và có thể làm tăng tải CPU/network.

```
1 // Initialize timer for 25 FPS (40ms)
2 frameTimer_ = std::make_unique<Timer>(10);
```

Listing 36: Interval Timer ở `ServerWorker` đang đặt 10ms dù comment nói 40ms

Khuyến nghị: đưa interval về 40ms hoặc đọc từ `Config` để đồng bộ với FPS mục tiêu và dễ hiệu chỉnh theo video.

7.2.0.2 (2) Logic `stop()` ở `ServerWorker` dùng static flag có thể gây lỗi khi có nhiều worker. Hàm `stop()` đang dùng `static std::atomic<bool> stopCalled` khiến chỉ worker đầu tiên được stop “thực sự”, các worker còn lại sẽ return sớm. Đây là rủi ro rõ ràng nếu server phục vụ nhiều client đồng thời.

```
1 void ServerWorker::stop() {
2     static std::atomic<bool> stopCalled{false};
3     if (stopCalled.exchange(true)) {
4         return;
5     }
6     // ...
7 }
```

Listing 37: Static stop flag có thể làm dừng sai khi nhiều worker

Khuyến nghị: biến `stopCalled` thành member per-instance hoặc bỏ hoàn toàn (vì bản thân `running_` và `streaming_` đã là cờ dừng).

7.2.0.3 (3) Tính năng tua (seek) trên timeline hiện mới là thay đổi chỉ số hiển thị, chưa có RTSP Range. Client cập nhật `currentFrame_` theo slider nhưng không gửi yêu cầu RTSP mới để server bắt đầu từ vị trí tương ứng; do đó “seek” hiện tại chỉ mang tính UI/telemetry.

```
1 void ClientUI::onTimelineSliderReleased() {
2     int targetFrame = timelineSlider_>value();
3     currentFrame_ = targetFrame;
4 }
```

Listing 38: Seek hiện tại chỉ cập nhật `currentFrame_` ở UI

Khuyến nghị: mở rộng RTSP với header Range (NPT hoặc frame index), và server cần hỗ trợ reset `VideoStream` đến offset phù hợp (có thể bằng index table hoặc scan marker nhanh).

7.2.0.4 (4) Một số chi tiết định dạng RTSP request nên được chuẩn hóa tuyệt đối.

Ví dụ, trong `RTSPClient::sendTeardown()` dòng request line thiếu khoảng trắng trước `RTSP/1.0`. Đây là lỗi định dạng có thể gây fail khi tương tác với server khác.

```
1 // Wrong:
2 ss << "TEARDOWN " << rtspUri_ << "RTSP/1.0\r\n";
3
4 // Correct:
5 ss << "TEARDOWN " << rtspUri_ << " RTSP/1.0\r\n";
```

Listing 39: Teardown request line thiếu khoảng trắng trước phiên bản

Khuyến nghị: thống nhất tất cả request thông qua `RTSPMessage::buildRequest()` để loại bỏ sai sót format.

7.3 Kết luận

Với các thành phần đã hiện thực, dự án đáp ứng yêu cầu nền tảng của một hệ thống streaming RTSP/RTP có khả năng phân mảnh frame lớn, quản lý session, và buffer hóa ở client. Đồng thời, do code đã có cấu trúc pattern rõ ràng (Builder/Strategy/Singleton) và ranh giới module sạch, hệ thống sẵn sàng để mở rộng sang các tính năng “production-grade” hơn như Range seek, codec khác, adaptive bitrate, và cơ chế đồng bộ A/V trong các phiên bản tiếp theo.

References

- [1] L. H. Minh, “Socket programming assignment: Rtp video streaming,” Course Material, 2024, document Socket_2526, Ho Chi Minh University of Science.
- [2] W. R. Stevens, B. Fenner, and A. M. Rudoff, *Unix Network Programming, Volume 1: The Sockets Networking API*, 3rd ed. Boston, MA: Addison-Wesley Professional, 2004.
- [3] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Boston, MA: Pearson, 2021, global Edition.
- [4] H. Schulzrinne, A. Rao, and R. Lanphier, “Real Time Streaming Protocol (RTSP),” IETF, RFC 2326, Apr. 1998. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc2326>
- [5] H. Schulzrinne, S. L. Casner, R. Frederick, and V. Jacobson, “RTP: A Transport Protocol for Real-Time Applications,” IETF, RFC 3550, Jul. 2003. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc3550>
- [6] A. M. Tekalp, *Digital Video Processing*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2015.
- [7] L. M. Berc, W. C. Fenner, R. Frederick, and S. McCanne, “RTP Payload Format for JPEG-compressed Video,” IETF, RFC 2435, Oct. 1998. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc2435>
- [8] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.